

A Major Project Report
on

NOESIS

An Operating-System-Inspired Kernel for Autonomous Multi-Agent Intelligence

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

School of Computer Science

Submitted by

Dhruv Shah (SAP ID 500118979)

Manan Nasa (SAP ID 500123471)

Under the guidance of

Dr. Archana Kumari



Department of Informatics

School of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES (UPES)

Dehradun – 248007, Uttarakhand, India

Academic Year 2026–2027 · 6 October 2026

Certificate

This is to certify that the project report entitled **“NOESIS: An Operating-System-Inspired Kernel for Autonomous Multi-Agent Intelligence”**, submitted by **Dhruv Shah (SAP ID 500118979)** and **Manan Nasa (SAP ID 500123471)** to the School of Computer Science, University of Petroleum & Energy Studies (UPES), Dehradun, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is a bona fide record of the work carried out by them under my supervision and guidance during the academic year 2026–2027.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Dr. Archana Kumari

Project Guide

School of Computer Science, UPES

Head of Department / Cluster

School of Computer Science, UPES

Date: _____

Place: Dehradun

Declaration

We hereby declare that the work presented in this report, entitled “**NOESIS: An Operating-System-Inspired Kernel for Autonomous Multi-Agent Intelligence**”, is our own work carried out under the guidance of Dr. Archana Kumari. The software described was designed and implemented by us; external libraries, frameworks, models and publications are acknowledged and cited. All quantitative results reported as *measured* were obtained by us on the stated environment, and all limitations we are aware of are disclosed. This work has not been submitted elsewhere for any degree or diploma.

Dhruv Shah

SAP ID 500118979

Manan Nasa

SAP ID 500123471

Acknowledgement

We express our sincere gratitude to our project guide, **Dr. Archana Kumari**, for her continuous guidance, critical feedback and encouragement throughout the design, implementation and evaluation of NOESIS. Her questions about evidence and reproducibility shaped the way this report separates measured results from design intent.

We thank the School of Computer Science, University of Petroleum & Energy Studies (UPES), for the academic environment and resources that made this project possible, and the faculty members and technical staff who supported us during the course of the work.

We acknowledge the open-source communities behind FastAPI, Pydantic, SQLAlchemy, Next.js, React, Ollama and the Qwen model family, whose tools made it possible to build and evaluate an agent kernel on student hardware. Finally, we thank our families, peers and friends for their encouragement and constructive discussions.

Dhruv Shah

Manan Nasa

Abstract

Frameworks for building multi-agent systems on large language models (LLMs) make it easy to wire prompts and tools together, but they usually leave permission boundaries, reproducibility, memory lifecycle and operational visibility to the application. This report presents **NOESIS**, an operating-system-inspired orchestration kernel that brings these concerns into the runtime. A goal submitted through a REST API or a web dashboard is decomposed by a deterministic, seed-controlled planner into a dependency graph of steps; a runtime executes the steps in topological order, each by one of twelve typed agents running under a freshly minted, least-privilege capability set; every step leaves an audit record; and a six-tier memory controller can promote entries while maintaining a SHA-256 provenance chain. A microkernel abstraction with 27 system-call identifiers, a capability table, a priority/deadline scheduler, a quota allocator, a weighted model router and an interrupt model underpins the design, and HMAC-signed capability tokens protect selected HTTP routes. LLMs are optional: when a local Ollama model is reachable its response to each agent's prompt is attached as rationale, otherwise the deterministic agents run alone.

The implementation comprises 17,072 lines of Python in the backend package, a Next.js 14 dashboard with 13 routes, and 42 HTTP operations. We evaluated it by executing its test suites and by measuring its behaviour on a laptop. All 310 non-skipped backend tests passed (2 expected failures) with 70.0 % line coverage; the planner produced byte-identical plans for all 50 goals of the project's SE-50 corpus across repeated runs in about 1 ms per plan; all capability-gate denial scenarios were rejected with specific reason codes; and with the 7-billion-parameter Qwen2.5-Coder model the system generated about 23 tokens per second, so a five-step LLM-enriched run took about one minute. The evaluation also shows the limits of the current build: agents are rule-based heuristics, resource routes are unauthenticated, document ingestion and the retrieval agent are simulated, six dashboard pages render placeholder data, and the code benchmarks check only that generated code compiles. We therefore position NOESIS as a reproducible, auditable kernel substrate whose agent intelligence is the principal subject of future work.

Keywords: multi-agent systems; large language models; capability-based security; deterministic planning; agent runtime; tiered memory; provenance; FastAPI; observability; reproducibility.

Table of Contents

Certificate.....	i
Declaration.....	ii
Acknowledgement	iii
Abstract.....	iv
List of Abbreviations	xiv
CHAPTER 1 INTRODUCTION	1
1.1 Background.....	1
1.2 Problem statement.....	1
1.3 Motivation.....	1
1.4 Research and engineering gap	2
1.5 Objectives	2
1.6 Scope.....	2
1.7 Contributions.....	3
1.8 Organisation of the report	3
CHAPTER 2 LITERATURE REVIEW	4
2.1 Reasoning and acting with language models	4
2.2 Multi-agent LLM frameworks	4
2.3 Operating-system abstractions for LLM agents.....	4
2.4 Capability-based security.....	5
2.5 Memory models and provenance	5
2.6 Retrieval-augmented generation	5
2.7 Evaluation of code-generating agents.....	5
2.8 Security of LLM-integrated applications.....	6
2.9 Comparative summary and gap	6
CHAPTER 3 REQUIREMENTS ANALYSIS.....	7
3.1 Stakeholders.....	7

3.2 Functional requirements.....	7
3.3 Non-functional requirements	9
3.4 Use cases.....	11
3.5 Constraints and assumptions.....	12
CHAPTER 4 SYSTEM DESIGN	13
4.1 Design principles	13
4.2 Architecture.....	13
4.3 Kernel design	14
4.4 Agent design	15
4.5 Data flow and runtime behaviour	16
4.6 Database design	18
4.7 API design.....	18
4.8 Algorithms	20
4.9 Security design.....	22
4.10 UI/UX design	23
CHAPTER 5 IMPLEMENTATION.....	24
5.1 Technology stack	24
5.2 Code base overview	25
5.3 Application bootstrap.....	25
5.4 Kernel and capabilities.....	25
5.5 Agents and services.....	26
5.6 Memory subsystem	26
5.7 Tools and sandboxing	27
5.8 LLM integration.....	27
5.9 Security implementation	27
5.10 Observability.....	27
5.11 Frontend	28

5.12 Evaluation harness	29
5.13 Deployment artefacts	30
CHAPTER 6 METHODOLOGY	31
6.1 Development methodology	31
6.2 Architecture methodology	31
6.3 Testing methodology	31
6.4 Evaluation methodology	31
6.5 Reproducibility controls.....	32
6.6 Deployment methodology.....	32
CHAPTER 7 RESULTS AND EVALUATION	33
7.1 Functional testing.....	33
7.2 Security and permission enforcement.....	34
7.3 Determinism and reproducibility	36
7.4 Performance	37
7.5 Memory promotion	39
7.6 Retrieval and benchmark harness	40
7.7 User-interface evaluation	40
7.8 Attainment of objectives	42
7.9 Comparative evaluation	43
CHAPTER 8 DISCUSSION.....	44
8.1 What worked and why	44
8.2 Trade-offs.....	44
8.3 Challenges and unexpected findings.....	44
8.4 Threats to validity	45
8.5 Limitations	45
CHAPTER 9 CONCLUSION AND FUTURE WORK	46
9.1 Summary.....	46

9.2 Contributions revisited.....	46
9.3 Achievement of objectives.....	46
9.4 Future work.....	46
References.....	48
Appendix A: Test Inventory	51
Appendix B: Sample Plan.....	52
Appendix C: Selected API Exchanges.....	53
Appendix D: Configuration Reference	55
Appendix E: Agent Codenames.....	56
Appendix F: Reconciliation of Earlier Project Claims	57
Appendix G: Reproduction Commands.....	58

List of Figures

Figure 3.1: Use-case diagram	11
Figure 4.1: Layered architecture	13
Figure 4.2: Component diagram	14
Figure 4.3: Kernel classes.....	15
Figure 4.4: Agent classes and roster	16
Figure 4.5: Level-1 data-flow diagram	16
Figure 4.6: Sequence of a conversation run.....	17
Figure 4.7: Per-step runtime with LLM enrichment and fallback	17
Figure 4.8: Entity–relationship diagram	18
Figure 4.9: API operations by route group	19
Figure 4.10: Deterministic planner algorithm.....	21
Figure 4.11: Executor tri-state decision and computed decision grid.....	21
Figure 4.12: Six-tier promotion pipeline and provenance chain	22
Figure 4.13: Capability-token verification sequence	22
Figure 4.14: Authentication sequence	23
Figure 4.15: Dashboard site map with data source per route.....	23
Figure 5.1: Backend source size by package (measured)	25
Figure 5.2: LLM provider factory, failover and adapters	27
Figure 5.3: Conversations page of the running dashboard (live data)	28
Figure 5.4: Thread view showing per-step agent reports (live data)	29
Figure 5.5: Deployment configuration.....	30
Figure 7.1: Backend test results per module (measured).....	33
Figure 7.2: Line coverage per package (measured)	34
Figure 7.3: Planner behaviour on the SE-50 corpus (measured)	36
Figure 7.4: In-process API latency (measured, deterministic mode)	37
Figure 7.5: Local-LLM latency and throughput through /llm/benchmark (measured)	38

Figure 7.6: Per-step duration of the same five-step plan without and with LLM enrichment (measured).....	38
Figure 7.7: Tier occupancy in a controlled promotion scenario (measured)	39
Figure 7.8: BM25 + RRF query latency over project documents (measured).....	40
Figure 7.9: Compile-pass per task in the W7 run (reported)	40
Figure 7.10: Dashboard (placeholder metrics) — screenshot of the running application	41
Figure 7.11: Agents page (placeholder fleet) — screenshot of the running application ..	41
Figure 7.12: Bench Results Hub with labelled demo data — screenshot of the running application.....	42
Figure 7.13: Execution timeline (placeholder traces) — screenshot of the running application.....	42

List of Tables

Table 1.1: Project objectives	2
Table 2.1: Qualitative comparison of concerns (characterisations of other systems are based on their publications/documentation)	6
Table 3.1: Stakeholders.....	7
Table 3.2: Functional requirement areas (65 requirements)	7
Table 3.3: Functional requirements index.....	7
Table 3.4: Implementation status of functional requirements.....	9
Table 3.5: Non-functional requirements	9
Table 3.6: Use cases.....	11
Table 4.1: Design principles	13
Table 4.2: System calls (27) and default handlers (6).....	15
Table 4.3: Database tables	18
Table 4.4: API operations (42).....	19
Table 5.1: Technology stack and rationale	24
Table 5.2: Agent implementations	26
Table 6.1: Evaluation questions and methods.....	31
Table 7.1: Test results (measured)	33
Table 7.2: Kernel syscall capability scenarios (measured)	34
Table 7.3: Capability-gate scenarios on HTTP routes (measured)	35
Table 7.4: Authentication scenarios (measured).....	35
Table 7.5: Reproducibility results.....	36
Table 7.6: API latency (measured, in-process, deterministic mode)	37
Table 7.7: GET /llm/benchmark runs against qwen2.5-coder:7b-instruct-q4_K_M (measured).....	37
Table 7.8: Benchmark results (reported, re-analysed)	40
Table 7.9: Achievement of objectives.....	42

Table 8.1: Design trade-offs.....	44
Table A.1: Backend test modules (measured)	51
Table D.1: Environment variables	55
Table E.1: Codename mappings found in the code	56
Table F.1: Claims versus implementation	57

List of Abbreviations

Abbreviation	Meaning
ABC	Abstract Base Class (Python)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AST	Abstract Syntax Tree
BM25	Okapi Best Matching 25 ranking function
CI	Continuous Integration
CI95	95 % confidence interval
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DFD	Data-Flow Diagram
DI	Dependency Injection
DTO	Data Transfer Object
ER	Entity–Relationship
FR	Functional Requirement
GC	Garbage Collection
HMAC	Hash-based Message Authentication Code
HS256	HMAC with SHA-256 (JWT algorithm)
HTTP(S)	Hypertext Transfer Protocol (Secure)
IRQ	Interrupt request (kernel interrupt record)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LOC	Lines of Code
LRU	Least Recently Used
MAC	Mandatory Access Control (as used by the project for capability gating)
MBPP	Mostly Basic Python Problems benchmark
NFR	Non-Functional Requirement
OOM	Out of Memory
ORM	Object–Relational Mapping
OTel	OpenTelemetry
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
RTM	Requirements Traceability Matrix
SE-50	Noesis software-engineering 50-task corpus
SHA-256	Secure Hash Algorithm, 256-bit
SQL	Structured Query Language
SRS	Software Requirements Specification
SSRF	Server-Side Request Forgery
TLV	Tag–Length–Value encoding
TTL	Time To Live

Abbreviation	Meaning
UAP	Universal Agent Protocol (project-defined)
UC	Use Case
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
WCAG	Web Content Accessibility Guidelines

CHAPTER 1

INTRODUCTION

1.1 Background

Large language models can follow natural-language instructions, reason step by step [10], interleave reasoning with actions [9], reflect on their own failures [11] and learn to call tools [12]. Building on these abilities, multi-agent frameworks assign roles to several model instances and let them cooperate: AutoGen orchestrates conversations between agents [13], MetaGPT encodes software-company procedures [14], CAMEL explores role-playing societies [15], and generative agents simulate believable behaviour with memory and reflection [16]. Surveys document the rapid growth of this field [17, 18].

Most of these systems are *libraries*: they provide abstractions for prompts, tools and agent conversations, and the application decides how agents are permitted to act, how results are recorded and how the system is operated. As soon as agents write files, execute code or call external services, these runtime concerns become central. They are exactly the concerns that operating systems address for processes — protection [2, 3], scheduling and lifecycle [5], and memory management. Two recent research lines make the analogy explicit: MemGPT treats the LLM context as virtual memory managed by an OS-like controller [7], and AIOS proposes an operating system that schedules and isolates LLM agents [8].

1.2 Problem statement

How can the primitives of an operating system — least-privilege capabilities, deterministic scheduling of work, tiered memory with provenance, and observable system calls — be applied to a multi-agent LLM runtime so that agent workflows become permission-checked, reproducible and auditable, while remaining usable on a single laptop with or without an LLM?

1.3 Motivation

- **Untrusted model output.** Prompt injection can turn retrieved text into instructions [27, 39]; side effects must therefore pass a permission check the model cannot bypass.
- **Reproducibility.** Debugging and regression testing require that the same input yields the same plan; sampling-based generation does not provide this by default.

- **Context limits.** Models use long contexts unevenly [22], motivating explicit memory tiers rather than ever-growing prompts.
- **Cost and privacy.** Local models such as Qwen2.5-Coder [26] can run on a laptop without per-token fees; a kernel should treat any model as an optional, replaceable component.
- **Operability.** Health probes, metrics, request correlation and audit logs are prerequisites for running agents unattended.

1.4 Research and engineering gap

Agent frameworks excel at composition but typically do not ship (i) a kernel-enforced capability check on every tool call and agent spawn, (ii) bit-exact replay of planning, (iii) a promotion-based memory hierarchy with an auditable provenance log, and (iv) an integrated operator console — together, in one self-hostable runtime. OS-for-LLM research (MemGPT, AIOS) focuses on context management and scheduling of LLM calls. NOESIS addresses the intersection: an engineering study of how far a conventional, test-driven Python service can realise these OS primitives for agents, and what remains missing. Chapter 2 develops this gap in detail.

1.5 Objectives

Table 1.1: Project objectives

ID	Objective	Where evaluated
O1	Design and implement a layered agent kernel with typed agents, a capability table and system-call tracing.	§7.1, §7.2
O2	Make planning deterministic and reproducible for equal (goal, seed) inputs.	§7.3
O3	Enforce least-privilege permissions for agents, tools and selected HTTP routes.	§7.2
O4	Provide tiered memory with a verifiable provenance chain.	§7.5
O5	Integrate optional LLM providers with graceful fallback, and characterise local-model performance.	§7.4
O6	Deliver a REST API and dashboard for running and monitoring workflows, with automated tests.	§7.1, §7.7
O7	Build an evaluation harness with statistical aggregation.	§7.6

1.6 Scope

The project covers the backend kernel, agents, memory, tools, LLM adapters, API, dashboard, tests and evaluation harness described in the companion SRS. It does not train models, does not implement distributed execution, and does not include the edge-hardware (NVIDIA Jetson) deployment that the synopsis proposed; that remains future work.

1.7 Contributions

The contributions of this work, stated with their evidence, are:

1. **An agent microkernel** with 27 system-call identifiers, per-agent capability tables with glob targets, OTel-shaped trace spans, a priority/deadline scheduler, a quota allocator with out-of-memory interrupts and a weighted model router (Chapter 5; 10 kernel tests).
2. **Least-privilege execution**: every plan step receives a token limited to its agent's declared capabilities plus its tool hints; the roster refuses under-privileged spawns and the tool registry refuses unlisted tools; HMAC-signed tokens with deny-mask precedence protect selected routes (measured: all denial scenarios rejected).
3. **Deterministic planning and audit**: a seed-controlled planner using UUIDv5 identities and a fixed epoch produces byte-identical plans (50/50 SE-50 goals) in about one millisecond.
4. **A six-tier memory promotion controller** with explicit triggers and a SHA-256 hash chain whose tail is reproducible (5/5 repetitions).
5. **LLM-optional design**: deterministic agents decide; an LLM, when available, contributes rationale text and token accounting, so the system degrades gracefully.
6. **An evidence-based evaluation and documentation package** that separates measured, derived and reported results and reconciles project claims with the implementation.

1.8 Organisation of the report

Chapter 2 reviews related work and identifies the gap. Chapter 3 analyses requirements. Chapter 4 presents the system design and Chapter 5 its implementation. Chapter 6 describes the development, testing and evaluation methodology. Chapter 7 reports results, Chapter 8 discusses them, and Chapter 9 concludes with future work. Appendices contain supporting material.

CHAPTER 2

LITERATURE REVIEW

2.1 Reasoning and acting with language models

Chain-of-thought prompting showed that eliciting intermediate reasoning improves multi-step problem solving [10]. ReAct interleaves reasoning traces with tool actions and observations, improving grounding and interpretability [9]. Reflexion adds verbal self-feedback stored in an episodic buffer to improve later attempts [11], and Toolformer trains a model to decide when to call APIs [12]. These works establish the *agent loop* — plan, act, observe, reflect — that NOESIS reproduces structurally with its planner, worker agents, reflection agent and executor gate. Unlike these approaches, NOESIS implements the loop's control logic deterministically and uses the model only for optional natural-language rationale.

2.2 Multi-agent LLM frameworks

AutoGen models applications as conversations among configurable agents, including humans and tools [13]. MetaGPT assigns software-engineering roles and standard operating procedures to agents to reduce cascading errors [14]. CAMEL studies cooperative role-playing between agents [15], and generative agents combine memory streams, reflection and planning to simulate behaviour [16]. Practitioner frameworks such as LangChain/LangGraph and CrewAI [46, 47] provide graphs, crews and tool abstractions. Across this body of work the emphasis is on *coordination* and *capability of agents*; permissioning, determinism and operations are generally delegated to the host application.

2.3 Operating-system abstractions for LLM agents

MemGPT frames the LLM as an operating system that pages information between a limited context window and external storage [7]. AIOS proposes an LLM agent operating system with scheduling, context switching, memory and tool management, and access control for concurrent agents [8]. NOESIS shares the analogy but differs in emphasis: it is a conventional web service whose kernel abstractions (syscalls, capability table, scheduler, allocator, interrupts) are implemented and tested as Python components, and whose central evaluation questions are reproducibility, permission enforcement and auditability rather than LLM throughput.

2.4 Capability-based security

Dennis and Van Horn introduced capabilities as unforgeable tickets that confer access rights [2]. Saltzer and Schroeder articulated design principles — least privilege, fail-safe defaults, complete mediation, economy of mechanism [3] — that remain the yardstick for access control. Miller, Yee and Shapiro clarified common misconceptions about capability systems [4]. NOESIS applies these ideas at two levels: an in-process capability table consulted by `kernel.syscall`, the roster and the tool registry (complete mediation for those paths, fail-safe default deny), and HMAC-signed bearer tokens [30] for HTTP routes, alongside conventional JWTs [31] for user identity. Section 7.2 evaluates both.

2.5 Memory models and provenance

The multi-store model of Atkinson and Shiffrin distinguishes sensory, short-term and long-term stores with transfer processes [1]; agent systems borrow this vocabulary for working, episodic and semantic memory [16, 7]. Hash-linked records, introduced for document time-stamping by Haber and Stornetta [29], provide tamper-evident histories. NOESIS's promotion controller combines both ideas: entries move through six tiers on explicit triggers, and each move is appended to a SHA-256 chain. Its T2→T3 test uses a lightweight fingerprint overlap in the spirit of similarity hashing [28].

2.6 Retrieval-augmented generation

Retrieval-augmented generation conditions a generator on retrieved passages [19]. Lexical ranking with BM25 remains a strong baseline [20], and reciprocal rank fusion combines rankings robustly without score calibration [21]. NOESIS implements parsing, chunking with citations, BM25 and RRF as a tested library; Chapter 7 reports its latency, and Chapter 8 discusses why it is not yet connected to the agent runtime.

2.7 Evaluation of code-generating agents

HumanEval introduced functional-correctness evaluation with the unbiased pass@k estimator [23]; MBPP provides basic Python problems [24]; SWE-bench evaluates resolution of real GitHub issues [25]. Reproducibility guidance for machine-learning research stresses fixed seeds, reported environments and released artefacts [35]; paired comparisons call for tests such as McNemar's [33], and uncertainty for bootstrap or score intervals [32, 34]. NOESIS implements these statistics; however, its harness currently checks compilation rather than executing tests, which Chapter 7 makes explicit.

2.8 Security of LLM-integrated applications

Indirect prompt injection shows that content fetched by an application can carry instructions to the model [27]; OWASP's LLM Top 10 lists prompt injection, insecure output handling and excessive agency among the main risks [39]. A kernel that mediates every side effect through capabilities limits *excessive agency* regardless of model behaviour.

2.9 Comparative summary and gap

Table 2.1: Qualitative comparison of concerns (characterisations of other systems are based on their publications/documentation)

Concern	Typical agent frameworks	OS-for-LLM research	NOESIS (as built)
Permission check on tool calls	Application-defined	Proposed access control (AIOS)	Capability check in ToolRegistry and Kernel.syscall (implemented)
Spawn-time privilege check	Rare	—	AgentRoster.spawn (implemented)
Reproducible planning	Depends on sampling	Not a focus	Seeded deterministic planner, UUIDv5 ids (implemented, measured)
Tiered memory with provenance	Vector stores / buffers	Paging of context (MemGPT)	Six tiers + SHA-256 chain (library; not driven in live runs)
LLM-optional operation	LLM required	LLM-centric	Deterministic agents; LLM adds rationale (implemented)
Operations (health, metrics, audit, console)	External	Partial	Probes, Prometheus, TaskExecution rows, dashboard (partially live)
Agent task quality	LLM-driven	LLM-driven	Rule-based heuristics (limitation)

Gap addressed. No single self-hostable runtime in the reviewed work combines kernel-mediated capabilities for tools and spawns, bit-exact plan replay, provenance-logged tiered memory and an operator console while remaining functional without an LLM. NOESIS contributes an implementation and an honest evaluation of that combination; it does not claim superior task performance to LLM-driven frameworks.

CHAPTER 3

REQUIREMENTS ANALYSIS

The requirements were derived from the implementation and documented in full in the companion Software Requirements Specification (SRS v1.0). This chapter summarises them.

3.1 Stakeholders

Table 3.1: Stakeholders

Stakeholder	Interest
Developers / researchers (dashboard users)	Run goals, inspect results, trust the audit trail
Operators	Health, metrics, configuration, safe defaults
Evaluators	Reproducible benchmark runs and statistics
Project guide / examiners	Evidence that objectives are met; honest limitations
Maintainers	Clear module boundaries, tests, extensibility

3.2 Functional requirements

Table 3.2: Functional requirement areas (65 requirements)

Area	Requirements	Status
Authentication and session tokens	FR-001, FR-002, FR-003, FR-004, FR-005	Implemented; Planned (not implemented)
Conversations and messages	FR-006, FR-007, FR-008, FR-009, FR-010, FR-011, FR-012	Implemented
Planning and plan execution	FR-013, FR-014, FR-015, FR-016, FR-017	Implemented; Partially implemented
Agent roster and agent behaviour	FR-018, FR-019, FR-020, FR-021, FR-022, FR-023, FR-024, FR-025, FR-026	Implemented; Stub / simulated
Documents	FR-027, FR-028, FR-029	Implemented; Stub / simulated
Memory records and tiered memory	FR-030, FR-031, FR-032, FR-033, FR-034, FR-035	Implemented; Implemented as library (not wired to API); Partially implemented; Planned (not implemented)
Kernel, capabilities and scheduling	FR-036, FR-037, FR-038, FR-039, FR-040, FR-041	Implemented; Implemented as library (not wired to API)
Tool registry and sandboxed tools	FR-042, FR-043, FR-044	Implemented; Implemented as library (not wired to API)
LLM provider integration	FR-045, FR-046, FR-047	Implemented; Implemented as library (not wired to API)
Health, observability and metrics	FR-048, FR-049, FR-050, FR-051	Implemented; Partially implemented; Stub / simulated
Benchmarking and evaluation	FR-052, FR-053, FR-054	Implemented; Partially implemented
Universal Agent Protocol	FR-055, FR-056	Implemented as library (not wired to API); Stub / simulated
Plugins and command-line interface	FR-057, FR-058	Implemented
Dashboard (Noesis Studio)	FR-059, FR-060, FR-061, FR-062, FR-063	Implemented; Partially implemented; Stub / simulated
Retrieval pipeline library	FR-064, FR-065	Implemented as library (not wired to API)

Table 3.3: Functional requirements index

ID	Requirement	Priority	Status
----	-------------	----------	--------

ID	Requirement	Priority	Status
FR-001	User login and token issuance	High	Implemented
FR-002	Identity introspection (/auth/me)	High	Implemented
FR-003	Access-token refresh	Medium	Implemented
FR-004	Production secret enforcement	High	Implemented
FR-005	Role-based authorisation on resource routes	High	Planned (not implemented)
FR-006	Create conversation from a goal	High	Implemented
FR-007	List, filter and search conversations	High	Implemented
FR-008	Read a conversation thread	High	Implemented
FR-009	Append message to a conversation	Medium	Implemented
FR-010	Update conversation (title, status, tags)	Medium	Implemented
FR-011	Delete conversation	Medium	Implemented
FR-012	List messages of a conversation	Low	Implemented
FR-013	Deterministic goal decomposition	High	Implemented
FR-014	Topological plan execution	High	Implemented
FR-015	Execution audit trail	Medium	Implemented
FR-016	List executions of a run	Medium	Partially implemented
FR-017	Background run completion messages	High	Implemented
FR-018	Agent catalogue	Medium	Implemented
FR-019	Direct single-agent invocation	Medium	Implemented
FR-020	Spawn-time capability gating of agents	High	Implemented
FR-021	Twelve typed agents	High	Implemented
FR-022	Research agent	Medium	Implemented
FR-023	Coding agent	Medium	Implemented
FR-024	Reflection, judge and critic agents	Medium	Implemented
FR-025	Executor tri-state decision	High	Implemented
FR-026	Memory, RAG, tool, supervisor and orchestrator agents	Low	Stub / simulated
FR-027	Register document metadata	Medium	Implemented
FR-028	Upload document file	Medium	Stub / simulated
FR-029	List, get and delete documents	Medium	Implemented
FR-030	List memory records	Medium	Partially implemented
FR-031	Query memory records	Low	Partially implemented
FR-032	Compress stale memory	Low	Implemented
FR-033	Six-tier memory promotion with provenance	High	Partially implemented
FR-034	Tier persistence across restarts	Medium	Planned (not implemented)
FR-035	Multi-tier recall bus	Low	Implemented as library (not wired to API)
FR-036	Kernel lifecycle and state	Medium	Implemented
FR-037	Capability-checked system calls	High	Implemented
FR-038	HMAC capability gate on HTTP routes	High	Implemented
FR-039	Kernel statistics and traces	Low	Implemented
FR-040	LLM enrichment with deterministic fallback	High	Implemented
FR-041	Scheduler, allocator, router and interrupts	Medium	Implemented as library (not wired to API)
FR-042	Capability-gated tool registry	High	Implemented
FR-043	Default sandboxed tools	High	Implemented
FR-044	Additional tool adapters	Low	Implemented as library (not wired to API)
FR-045	Pluggable LLM providers	High	Implemented
FR-046	Provider failover	Medium	Implemented
FR-047	Embedding providers	Low	Implemented as library (not wired to API)

ID	Requirement	Priority	Status
FR-048	Liveness and readiness probes	High	Implemented
FR-049	Prometheus and JSON metrics	Medium	Partially implemented
FR-050	Observability summary	Low	Stub / simulated
FR-051	Request correlation and structured logs	Medium	Implemented
FR-052	LLM throughput benchmark endpoint	Medium	Implemented
FR-053	Benchmark results API	Medium	Implemented
FR-054	Offline evaluation harness	Medium	Partially implemented
FR-055	UAP message model and wire codecs	Low	Implemented as library (not wired to API)
FR-056	UAP HTTP endpoints	Low	Stub / simulated
FR-057	Plugin discovery and lifecycle	Low	Implemented
FR-058	Command-line interface	Low	Implemented
FR-059	Dashboard shell and navigation	Medium	Implemented
FR-060	Conversation workflow in the UI	High	Implemented
FR-061	Monitoring pages (dashboard, agents, timeline, memory, documents, observability)	Medium	Stub / simulated
FR-062	Benchmark hub and runner pages	Medium	Partially implemented
FR-063	Settings page	Low	Stub / simulated
FR-064	Document parsing and chunking	Medium	Implemented as library (not wired to API)
FR-065	Hybrid lexical search	Medium	Implemented as library (not wired to API)

Table 3.4: Implementation status of functional requirements

Status	Count	Share
Implemented	43	66 %
Partially implemented	7	11 %
Implemented as library (not wired to API)	7	11 %
Stub / simulated	6	9 %
Planned (not implemented)	2	3 %

Of 65 functional requirements, 43 are fully implemented, 7 partially, 7 exist as libraries not yet wired to the API, 6 are stubs or simulations and 2 are planned. The partially implemented and stub items are concentrated in documents, memory, dashboard monitoring pages and observability; the core run pipeline, authentication, capability gating and kernel are implemented.

3.3 Non-functional requirements

Table 3.5: Non-functional requirements

ID	Category	Requirement	Status
NFR-PERF-01	Performance	Lightweight read endpoints shall respond quickly in-process.	Met
NFR-PERF-02	Performance	Planning shall add negligible latency compared with LLM inference.	Met
NFR-PERF-03	Performance	Conversation creation shall return before the run completes.	Met
NFR-PERF-04	Performance	Plan/execute endpoints should not pay a provider-probe cost on every request.	Not met

ID	Category	Requirement	Status
NFR-PERF-05	Performance	Local LLM throughput shall be characterised.	Met (characterised)
NFR-SCAL-01	Scalability	List endpoints shall paginate.	Met
NFR-SCAL-02	Scalability	The backend should support horizontal scaling.	Not met
NFR-REL-01	Reliability	A failure in one plan step shall not abort the run.	Met
NFR-REL-02	Reliability	LLM unavailability shall not break execution.	Met
NFR-REL-03	Reliability	Transient provider errors shall be retried.	Met
NFR-AVL-01	Availability	Optional dependencies shall not make the service unhealthy.	Met
NFR-AVL-02	Availability	Container health checks shall be defined.	Not verified
NFR-SEC-01	Security	Credentials shall be stored as salted slow hashes.	Met
NFR-SEC-02	Security	Tokens shall be integrity-protected and short-lived.	Met
NFR-SEC-03	Security	Resource APIs shall require authentication.	Not met
NFR-SEC-04	Security	Brute-force login shall be throttled.	Met
NFR-SEC-05	Security	Untrusted code/tool execution shall be contained.	Partially met (no SSRF filter; sandbox thread cannot be killed)
NFR-PRIV-01	Privacy	Secrets shall not appear in logs.	Met
NFR-PRIV-02	Privacy	Users shall only see their own data.	Not met
NFR-MNT-01	Maintainability	Code shall follow a ports-and-adapters structure with typed DTOs.	Met
NFR-MNT-02	Maintainability	Automated tests shall gate quality.	Met
NFR-USA-01	Usability	The UI shall give feedback for long-running work and errors.	Met
NFR-ACC-01	Accessibility	The UI should meet WCAG 2.1 AA.	Not verified
NFR-PORT-01	Portability	The backend shall run on Windows, Linux and macOS with SQLite by default.	Met (Windows measured)
NFR-PORT-02	Portability	The relational store should be switchable to PostgreSQL by URL.	Partially met
NFR-COMP-01	Compatibility	Python and Node versions shall be declared.	Met
NFR-OBS-01	Observability	Requests shall be traceable.	Met
NFR-OBS-02	Observability	Agent steps shall be measurable.	Not met
NFR-FT-01	Fault tolerance	Unhandled exceptions shall not leak internals.	Met
NFR-REC-01	Recoverability	In-memory state should be recoverable after restart.	Not met
NFR-RES-01	Resource utilisation	Kernel buffers shall be bounded.	Met
NFR-RES-02	Resource utilisation	Uploads should be bounded.	Not met
NFR-DET-01	Reproducibility	Equal (goal, seed) inputs shall produce bit-identical plans.	Met

3.4 Use cases

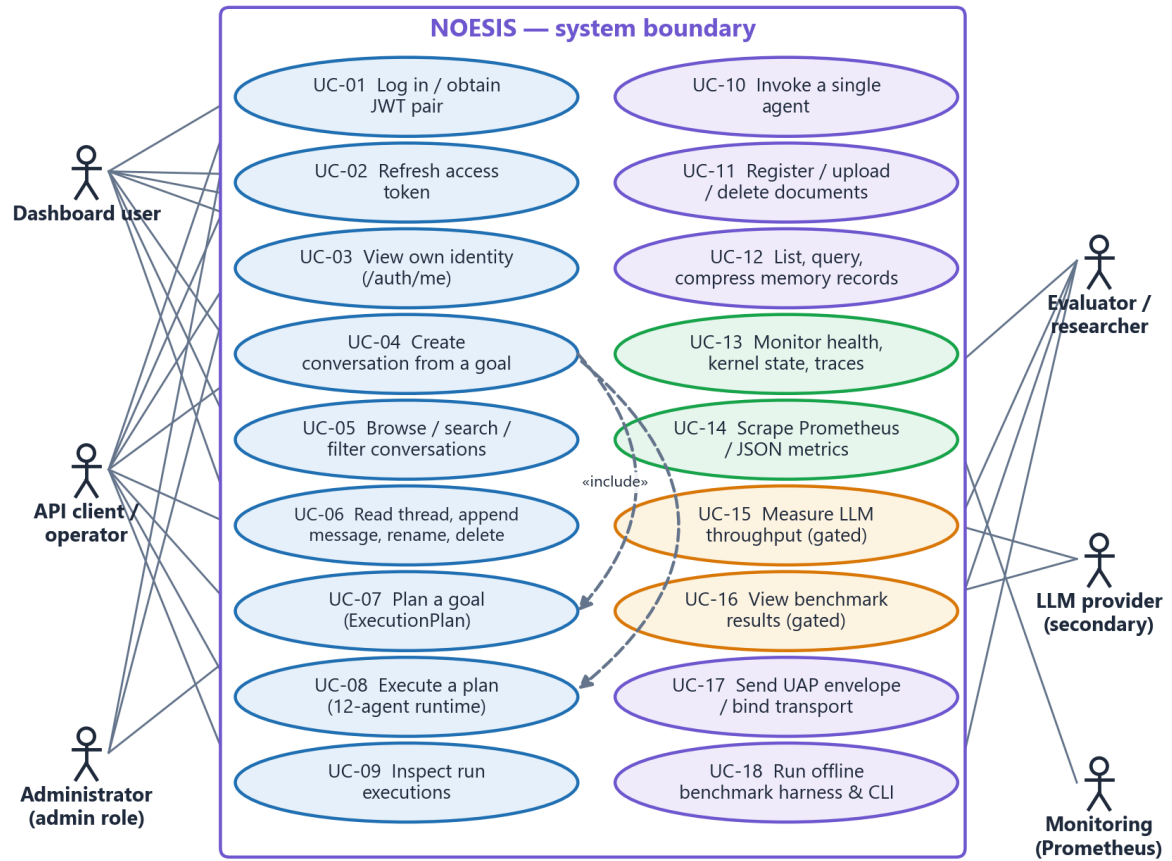


Figure 3.1: Use-case diagram

Table 3.6: Use cases

ID	Use case	Primary actor	Requirements
UC-01	Log in and obtain tokens	Dashboard user / API client	FR-001
UC-02	Refresh access token	API client	FR-003
UC-03	View own identity	Authenticated client	FR-002
UC-04	Start an autonomous run from a goal	Dashboard user	FR-006, FR-013, FR-014, FR-017, FR-040, FR-060
UC-05	Browse and search conversations	Dashboard user	FR-007, FR-060
UC-06	Work with a thread	Dashboard user / API client	FR-008 – FR-012
UC-07	Plan a goal	API client	FR-013
UC-08	Execute a plan	API client	FR-014, FR-015, FR-040
UC-09	Inspect run executions	API client / Operator	FR-016
UC-10	Invoke a single agent	API client	FR-019
UC-11	Manage documents	Dashboard user / API client	FR-027 – FR-029
UC-12	Query and compact memory records	API client	FR-030 – FR-032
UC-13	Monitor health and kernel	Operator / Administrator	FR-036, FR-039, FR-048
UC-14	Scrape metrics	Prometheus	FR-049
UC-15	Measure LLM throughput	Evaluator	FR-038, FR-052
UC-16	View benchmark results	Evaluator	FR-053, FR-062
UC-17	Exchange UAP envelopes	Integrator	FR-056
UC-18	Run offline evaluation and CLI	Evaluator / Developer	FR-054, FR-058

3.5 Constraints and assumptions

- Single-process deployment on a developer machine; SQLite by default.
- LLMs reached only through HTTP; any provider may be absent.
- English-language goals (keyword tables).
- Trusted network: resource routes are unauthenticated in the current build.
- Determinism must not depend on wall-clock time or Python's randomised `hash()`.

CHAPTER 4

SYSTEM DESIGN

4.1 Design principles

Table 4.1: Design principles

Principle	Design decision	OS analogy
Least privilege / fail-safe default	Agents and tools are denied unless a capability allows the (operation, target) pair	Capability lists, deny-by-default
Complete mediation (for kernel paths)	All side effects of agents go through ToolRegistry.invoke or Kernel.syscall	System-call boundary
Determinism by construction	Seeds, UUIDv5 identities, fixed epoch, sorted iteration, no hash()	Reproducible builds
Separation of decision and generation	Rule-based agents decide; LLM text is advisory	Policy vs mechanism
Ports and adapters	Domain code depends on ABC ports; adapters for SQL/LLM/stores [36]	Device drivers
Observability first	Request ids, spans, audit rows, metrics	/proc, syslog

4.2 Architecture

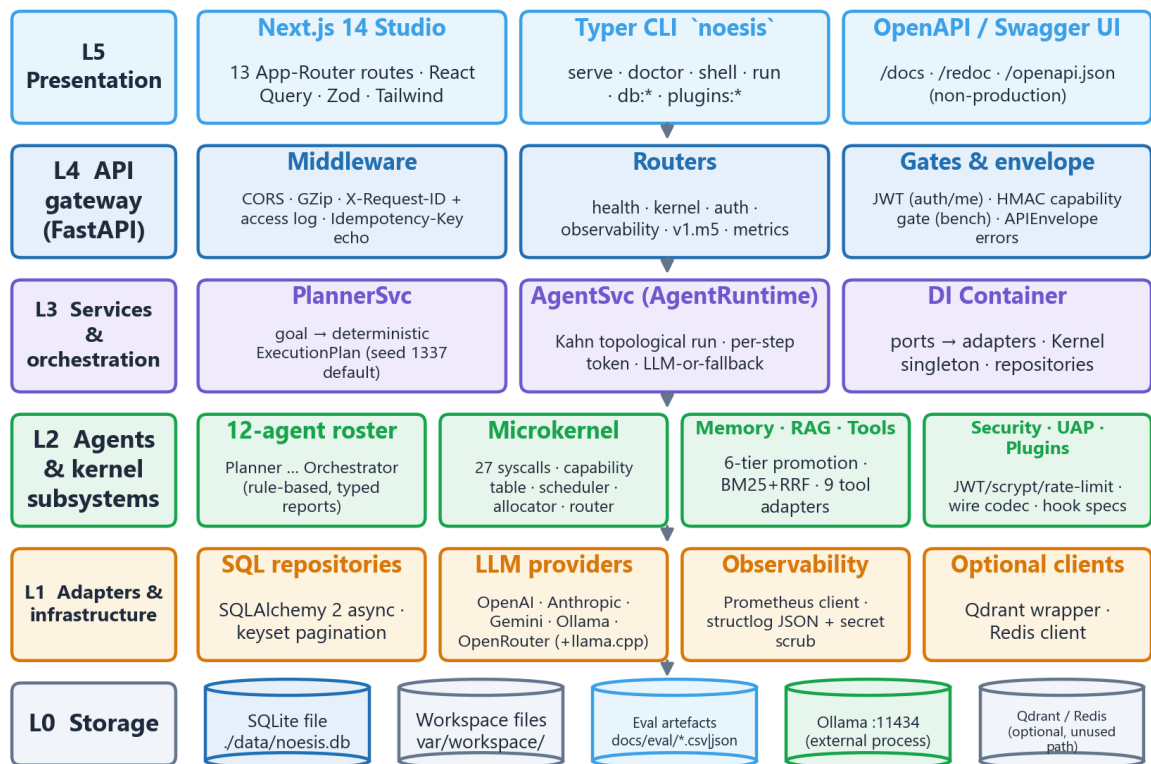


Figure 4.1: Layered architecture

Five layers over storage; dependencies point downwards only.

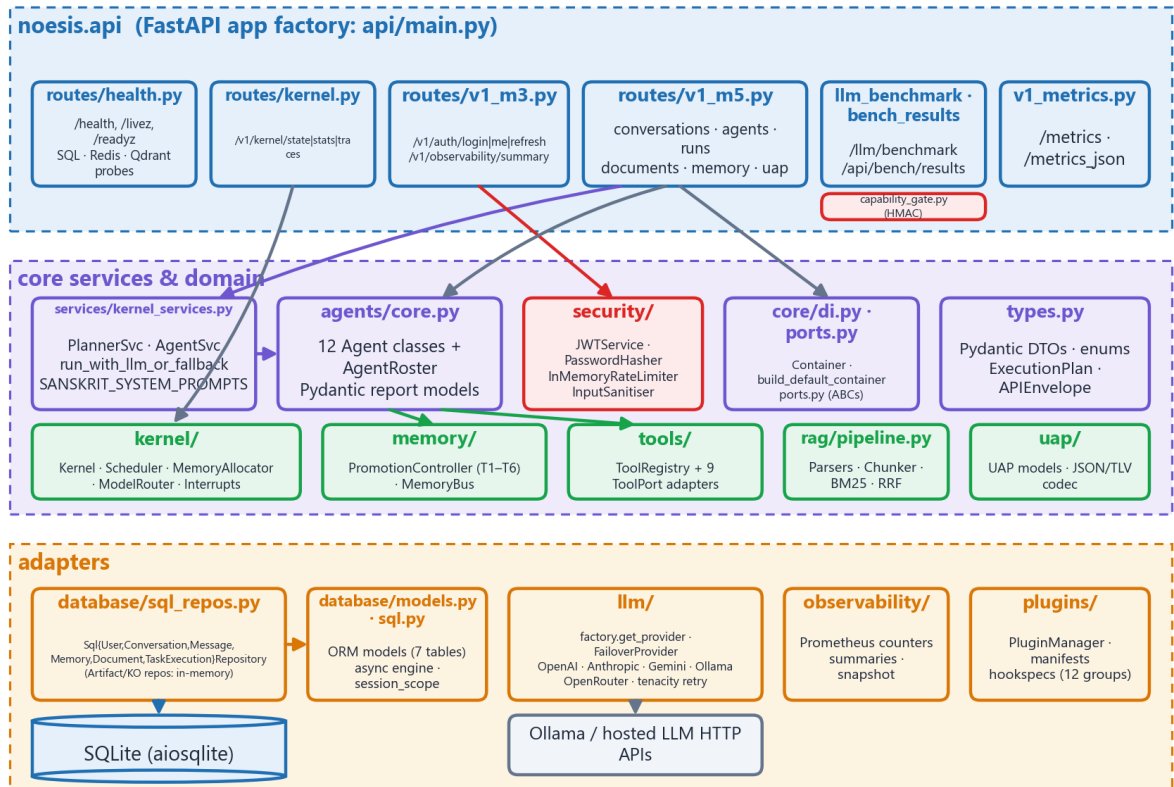


Figure 4.2: Component diagram

Backend modules and their principal dependencies.

4.3 Kernel design

The kernel is the trust boundary for agents. Each spawned agent receives an `AgentHandle` (agent id, run id); the kernel stores the authoritative `AgentContext` and capability list. A syscall is mapped to a capability operation and target (for example `TOOL_INVOKE` on the tool name, `MEMORY_WRITE` on the zone), checked against the list with glob matching, dispatched to a registered handler, and recorded as a span. The scheduler, allocator and model router are kernel-owned services.

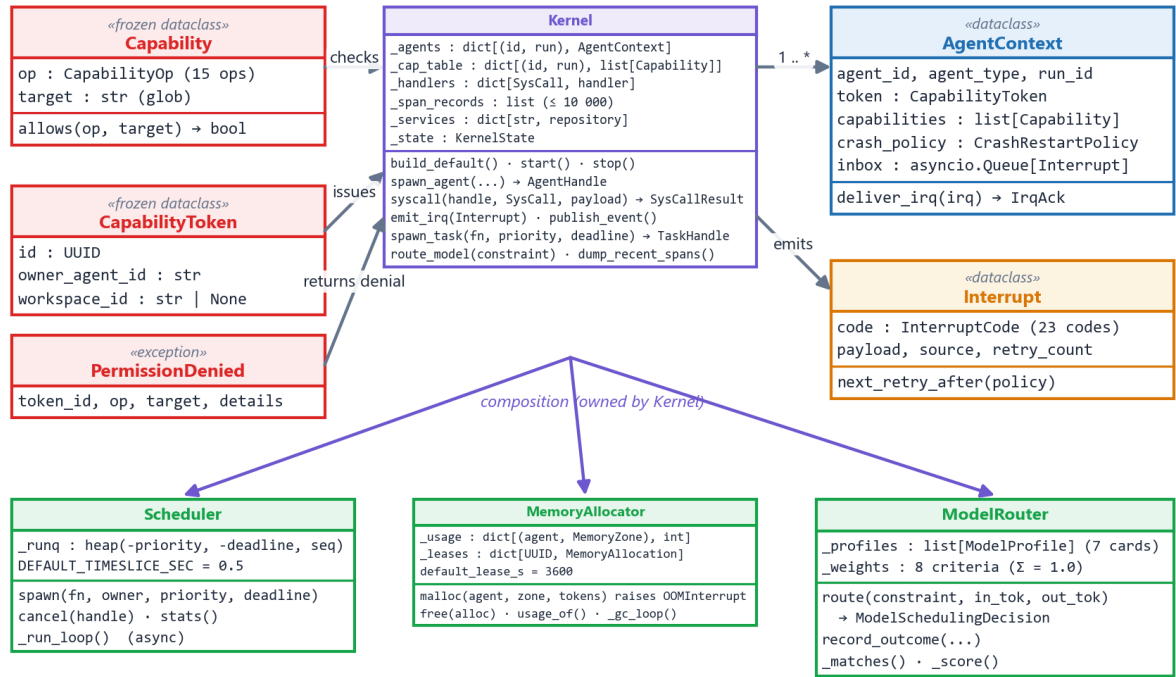


Figure 4.3: Kernel classes

Table 4.2: System calls (27) and default handlers (6)

Syscall group	Identifiers	Default handler
Memory	MEMORY_READ, MEMORY_WRITE, MEMORY_SEARCH, MEMORY_PRUNE	READ (repository), WRITE (allocator)
Knowledge	RAG_SEARCH, RAG_INGEST	—
Model	MODEL_CHAT, MODEL_EMBED, MODEL_ROUTE	CHAT, ROUTE
Tools	TOOL_INVOKE, TOOL_REGISTER, TOOL_DESCRIBE	INVOKE
Communication	SEND_MESSAGE, BROADCAST_EVENT, SUBSCRIBE_EVENTS	—
Lifecycle	SPAWN_AGENT, KILL_AGENT, SUSPEND, RESUME	—
Workspace	WORKSPACE_READ, WORKSPACE_WRITE	—
Meta	TRACE_EVENT, GET_CLOCK, GET_RUN_CONTEXT	GET_CLOCK
Plugins	PLUGIN_LOAD, PLUGIN_CALL	—

4.4 Agent design

Every agent implements `run(state, ctx) -> state` and returns a typed report; this LangGraph-style dictionary contract keeps agents composable and easy to test. Required capabilities are declared as class attributes so the runtime can mint a minimal token before execution.

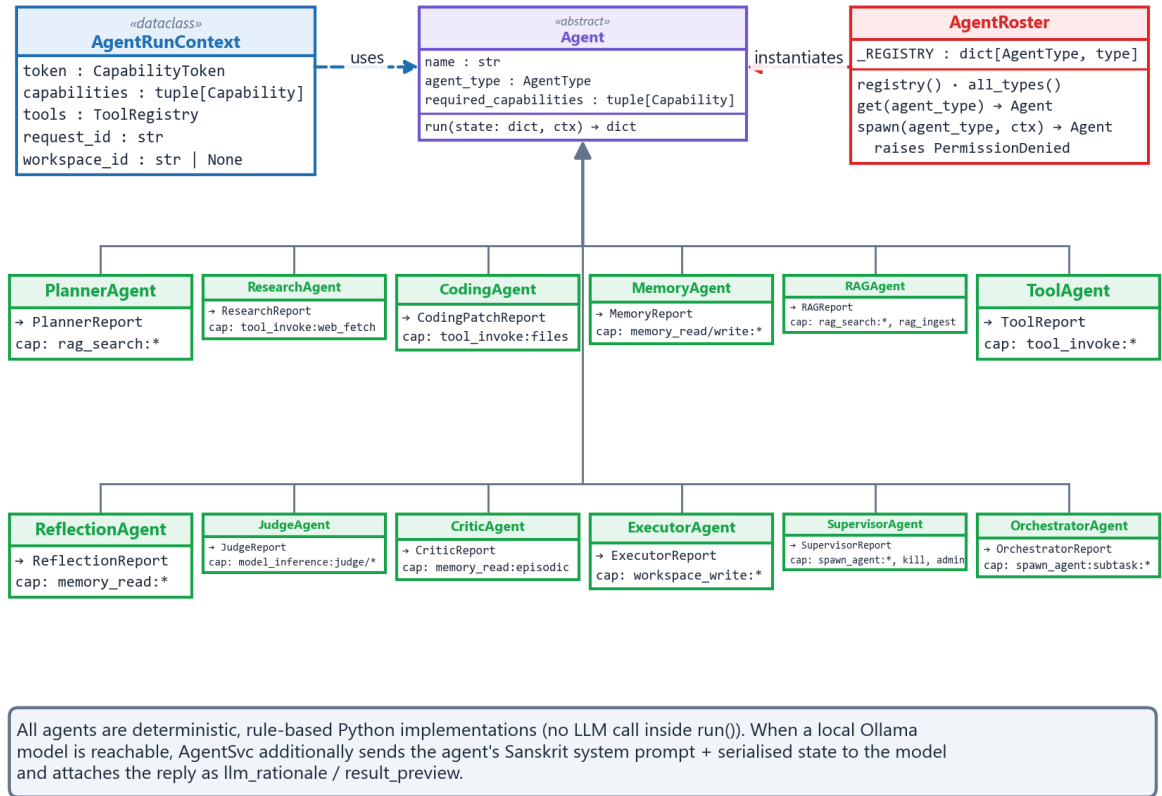


Figure 4.4: Agent classes and roster

4.5 Data flow and runtime behaviour

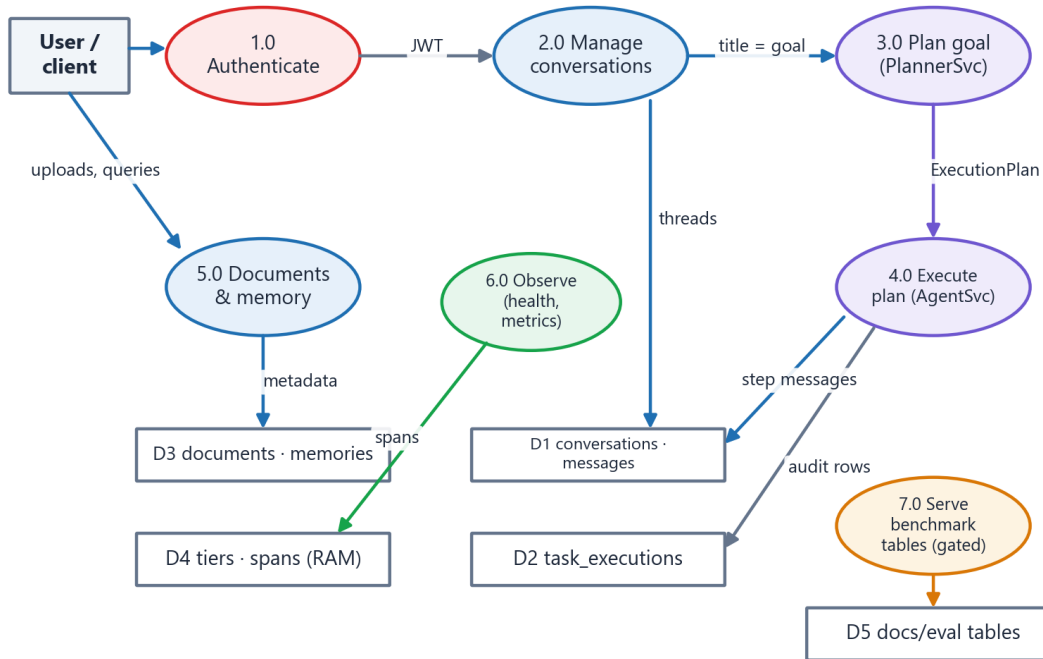


Figure 4.5: Level-1 data-flow diagram

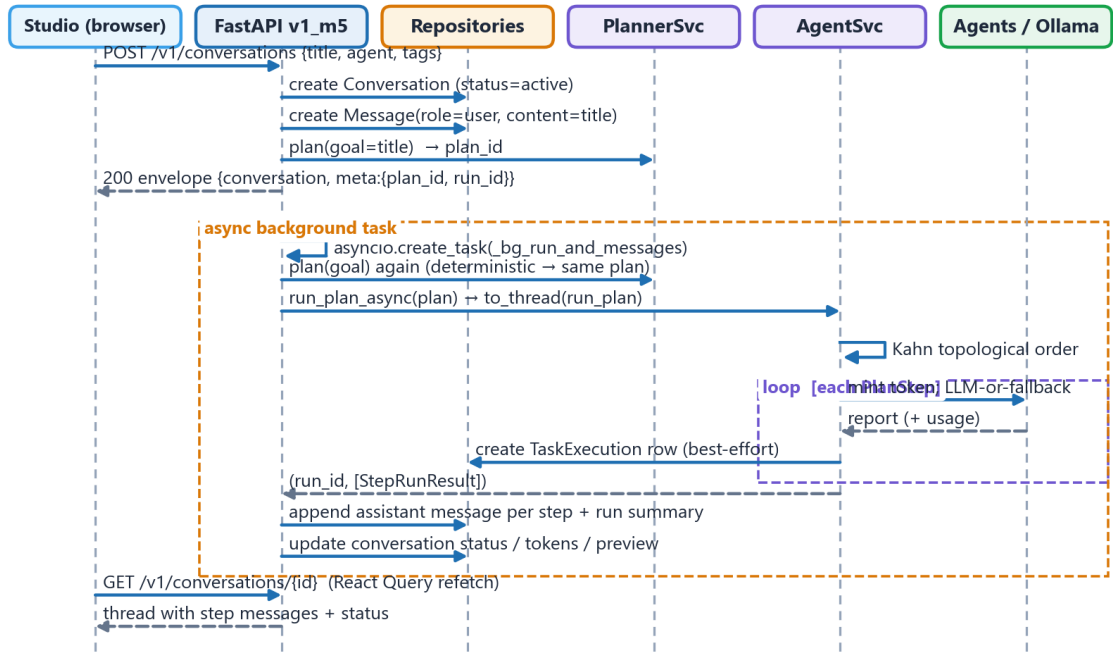


Figure 4.6: Sequence of a conversation run

Creation returns immediately; planning, execution and message writing happen in a background task.

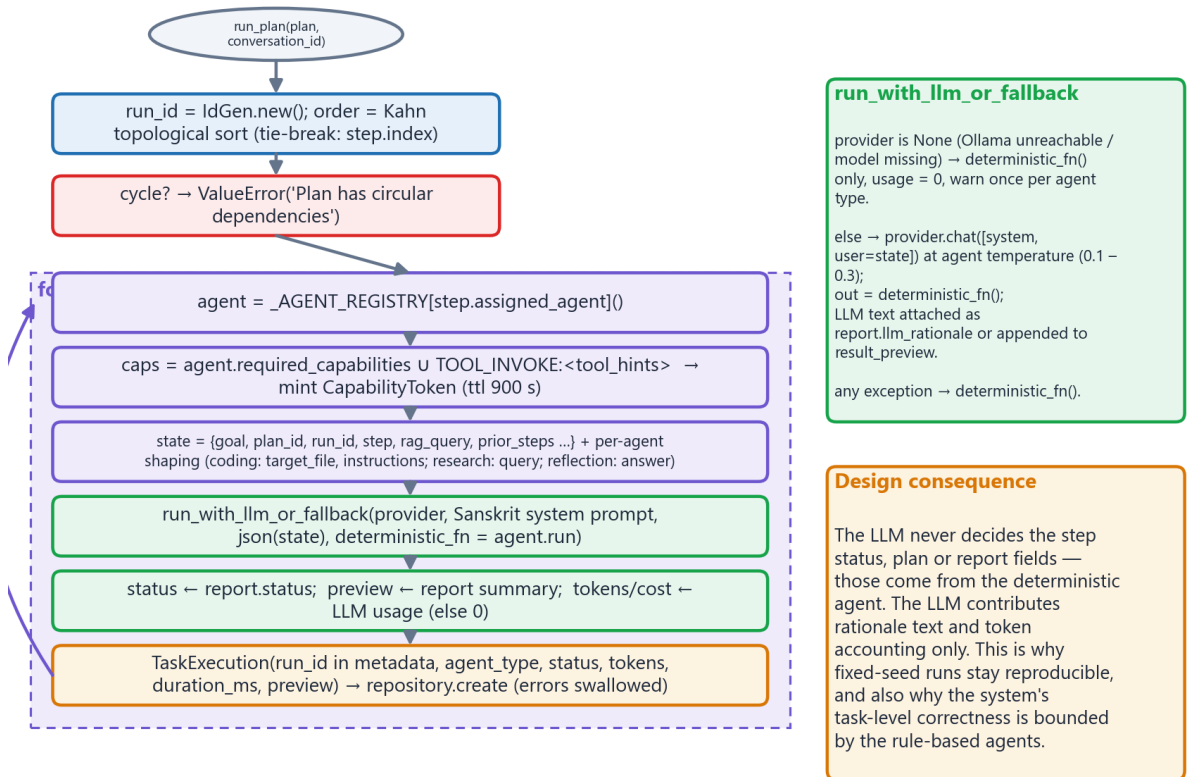


Figure 4.7: Per-step runtime with LLM enrichment and fallback

4.6 Database design

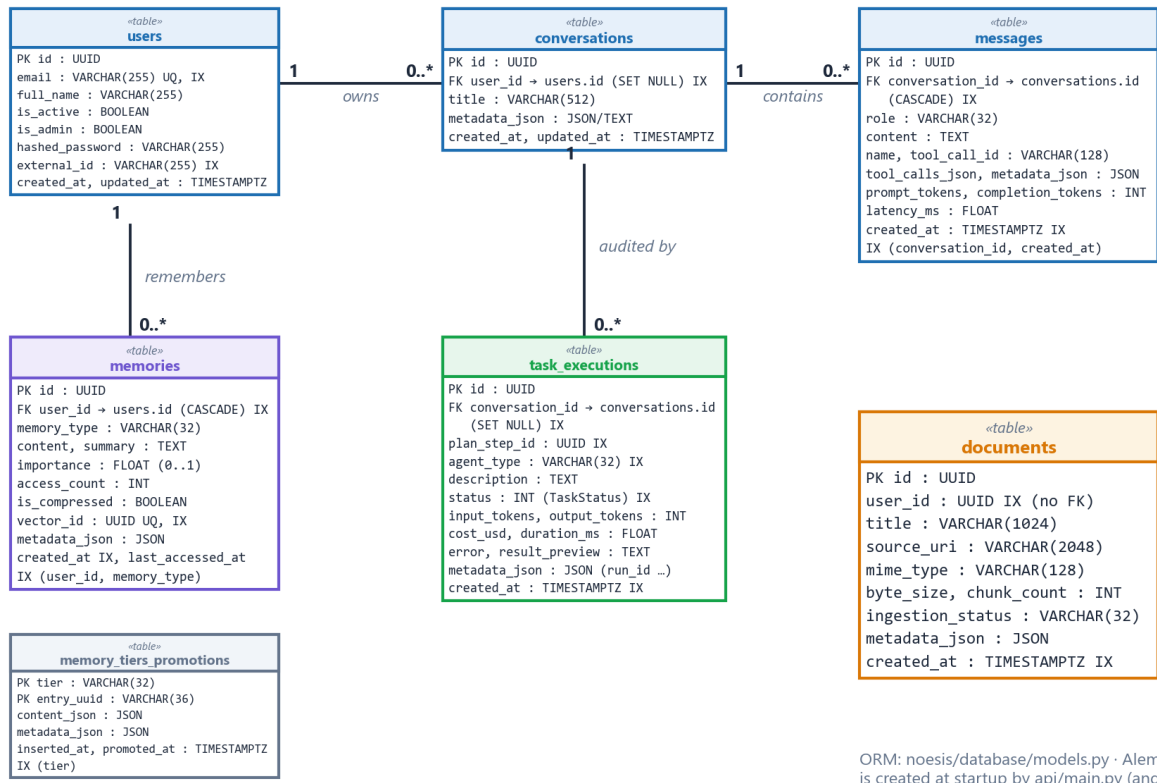


Figure 4.8: Entity–relationship diagram

Seven tables; keyset pagination on UUID primary keys; JSON metadata columns portable between SQLite and PostgreSQL.

Table 4.3: Database tables

Table	Columns	Primary key	Foreign keys	Purpose
documents	10	id	—	Document metadata
memory_tiers_promotions	6	tier, entry_uuid	—	Tier snapshot (unused)
users	9	id	—	User accounts (unused by login)
conversations	6	id	users.id ON DELETE SET NULL	Goal threads
memories	12	id	users.id ON DELETE CASCADE	Long-term memory rows
messages	12	id	conversations.id ON DELETE CASCADE	Thread turns
task_executions	14	id	conversations.id ON DELETE SET NULL	Per-step audit trail

4.7 API design

The API exposes 42 operations under `/v1` (plus root aliases for probes and metrics). All responses use the envelope `{ok, data, error, request_id, meta}`, validation errors are returned as 422 with structured locations, and list endpoints use cursor pagination. Two routes are protected by HMAC capability tokens; identity endpoints use JWTs.

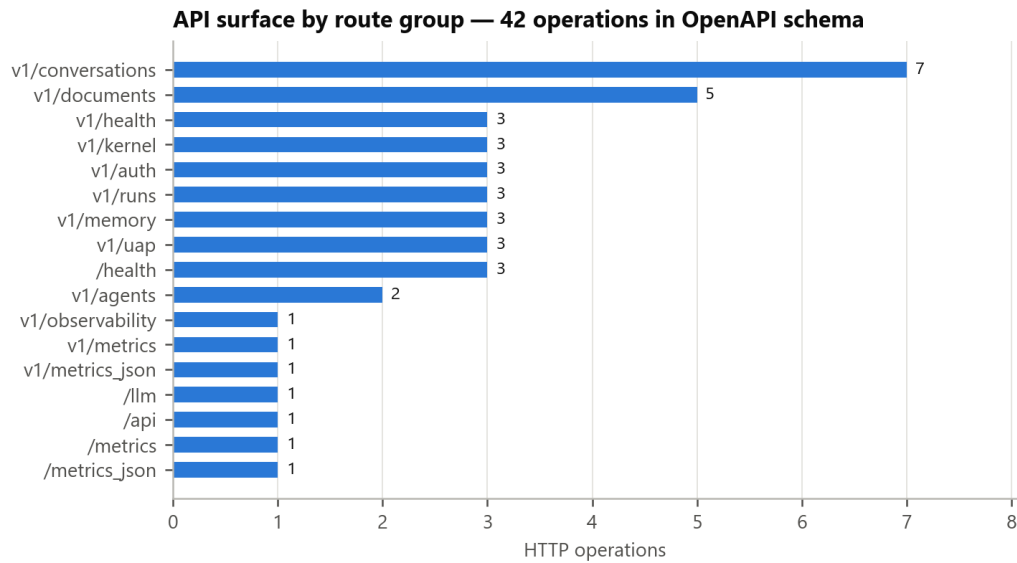


Figure 4.9: API operations by route group

Table 4.4: API operations (42)

Method	Path	Purpose	Auth
GET	/v1/health/livez	Liveness probe mounted under /v1.	open
GET	/v1/health/readyz	Readiness probe mounted under /v1.	open
GET	/v1/health	Readiness alias mounted under /v1.	open
GET	/v1/kernel/state	Kernel lifecycle state, uptime, registered services/handlers and span count.	open
GET	/v1/kernel/stats	Scheduler, memory-allocator and model-router aggregates.	open
GET	/v1/kernel/traces	Most recent syscall spans (OTel-shaped).	open
POST	/v1/auth/login	Authenticate and issue an access/refresh JWT pair.	open
GET	/v1/auth/me	Return the claims of the presented access token.	Bearer access JWT
POST	/v1/auth/refresh	Exchange a refresh token for a new access token.	Bearer refresh JWT
GET	/v1/observability/summary	Window summary of request latency, tokens, cost and tool traces (in-memory rings).	open
GET	/v1/conversations	Paginated, filterable conversation list.	open
POST	/v1/conversations	Create a conversation from a goal and start the background run.	open
GET	/v1/conversations/{conversation_id}	Fetch one conversation with optional messages.	open
PATCH	/v1/conversations/{conversation_id}	Update title, status or tags.	open
DELETE	/v1/conversations/{conversation_id}	Delete a conversation (messages cascade).	open
GET	/v1/conversations/{conversation_id}/messages	Page through messages of a conversation.	open
POST	/v1/conversations/{conversation_id}/messages	Append a message and update counters/status.	open
GET	/v1/agents	List the built-in agent manifests.	open
POST	/v1/agents/invoke	Run planner, research, coding or reflection on a state dict.	open
POST	/v1/runs/plan	Deterministically decompose a goal into an ExecutionPlan.	open
POST	/v1/runs/execute	Execute a plan step by step and return per-step results.	open
GET	/v1/runs/{run_id}/executions	List TaskExecution rows (filter uses conversation_id — see FR-016).	open
GET	/v1/documents	List document records with status and search filters.	open
POST	/v1/documents	Create document metadata (status pending).	open
GET	/v1/documents/{document_id}	Fetch one document record.	open
DELETE	/v1/documents/{document_id}	Delete a document record.	open
POST	/v1/documents/upload	Multipart upload; records size/type and a simulated chunk count.	open

Method	Path	Purpose	Auth
GET	/v1/memory	List long-term memory rows.	open
POST	/v1/memory/query	Rank memory rows by Jaccard token overlap + 0.1·importance.	open
POST	/v1/memory/compress	Flag rows older than N days as compressed.	open
GET	/v1/uap/transport	Static list of UAP transports.	open
POST	/v1/uap/transport/inproc	Return an in-process binding descriptor and random token string.	open
POST	/v1/uap/envelope/send	Accept an envelope and return its payload hash (no delivery).	open
GET	/v1/metrics	Prometheus exposition (same as /metrics).	open
GET	/v1/metrics_json	JSON snapshot of all counters and summaries.	open
GET	/health/livez	Liveness probe; always reports pass while the process runs.	open
GET	/health/readyz	Readiness probe: SQL (hard), Redis and Qdrant (soft) checks run concurrently.	open
GET	/health	Alias of /health/readyz.	open
GET	/llm/benchmark	Measure LLM latency and tokens/s against Ollama (mock fallback).	X-Noesis-Capability-Token with capability <code>llm.benchmark.read</code>
GET	/api/bench/results	Serve the newest benchmark tables (demo payload if none).	X-Noesis-Capability-Token with capability <code>bench.results.read</code>
GET	/metrics	Prometheus scrape endpoint.	open
GET	/metrics_json	JSON metrics snapshot (root mount).	open

4.8 Algorithms

The principal algorithms — deterministic planning, topological execution, promotion with hash chaining, the tri-state executor decision, capability-token verification, weighted model routing, priority scheduling, BM25 with RRF and sliding-window rate limiting — are specified with pseudocode in Chapter 13 of the SRS. Figure 4.10, Figure 4.11 and Figure 7.7 show the three that define system behaviour most directly.

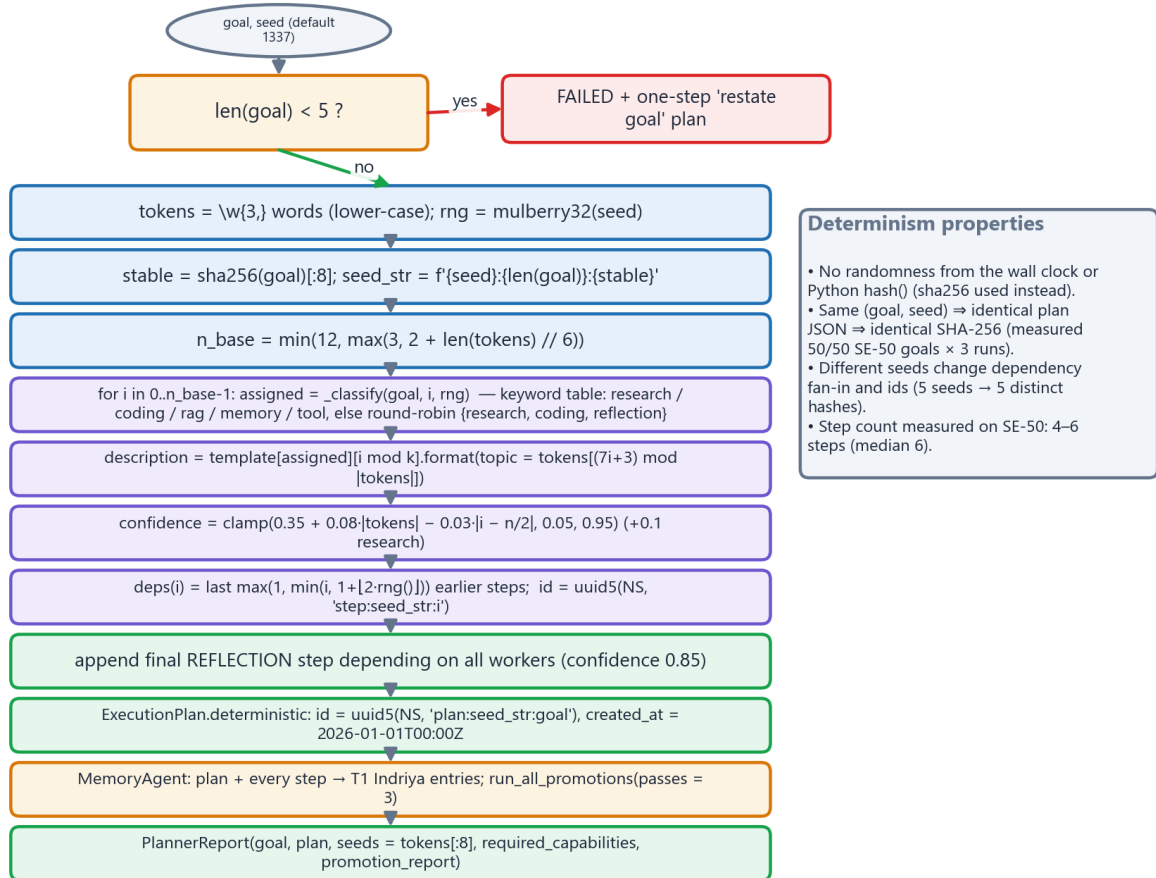


Figure 4.10: Deterministic planner algorithm

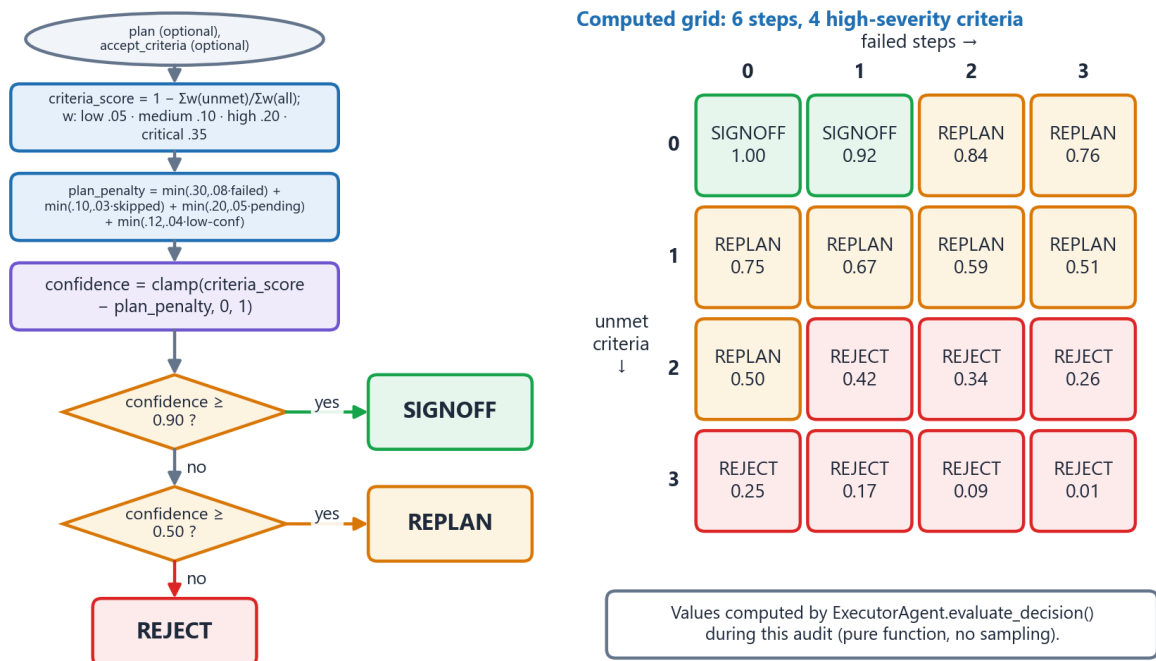


Figure 4.11: Executor tri-state decision and computed decision grid

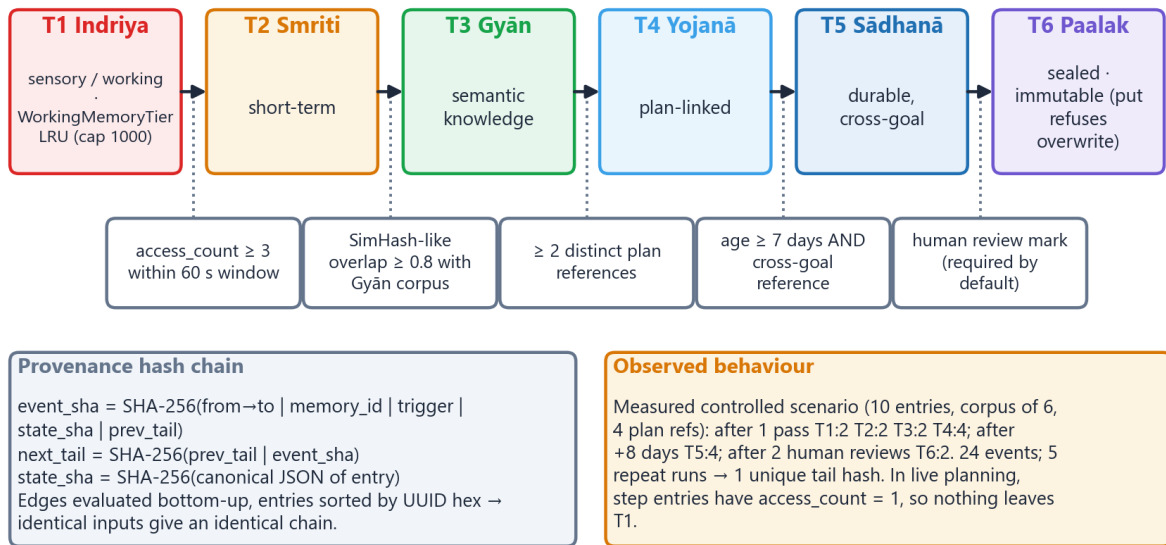


Figure 4.12: Six-tier promotion pipeline and provenance chain

4.9 Security design

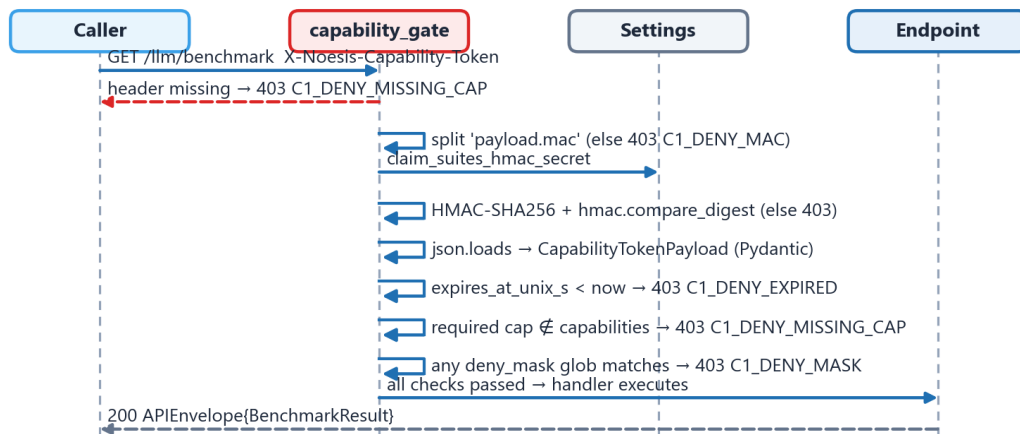


Figure 4.13: Capability-token verification sequence

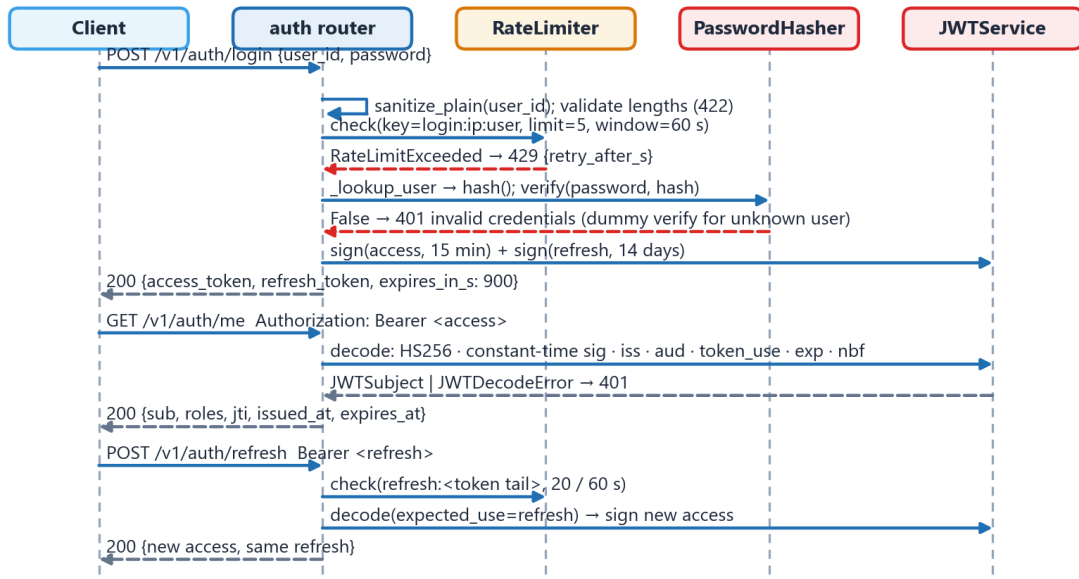


Figure 4.14: Authentication sequence

4.10 UI/UX design

The dashboard follows a console layout: persistent navigation, KPI tiles, timelines and toasts. Agent codenames are rendered as coloured badges to make multi-agent threads scannable.

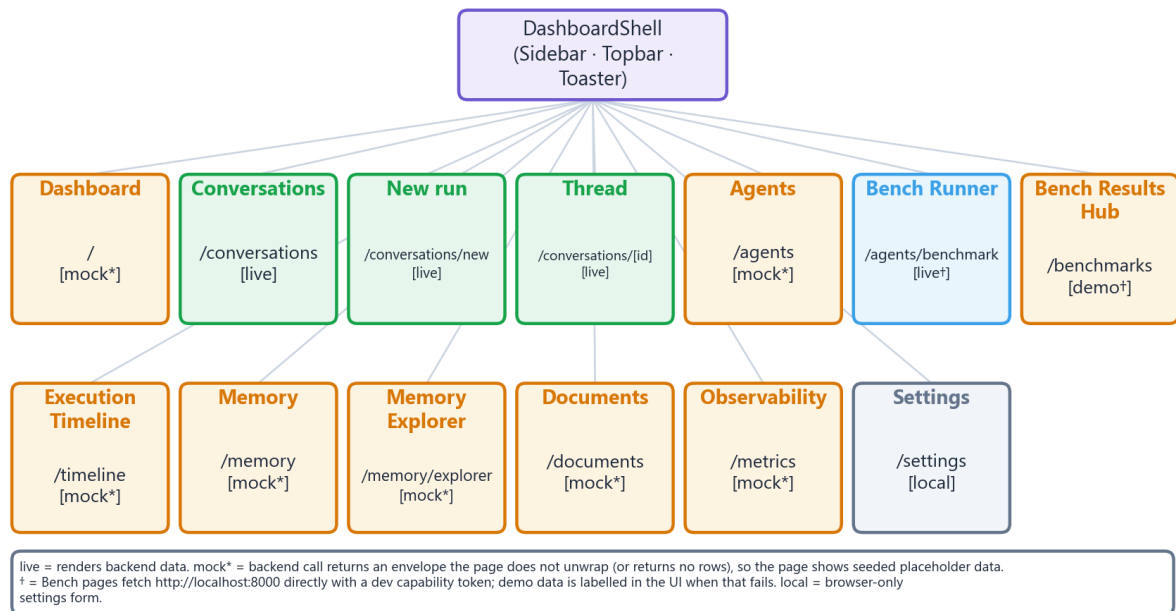


Figure 4.15: Dashboard site map with data source per route

CHAPTER 5

IMPLEMENTATION

5.1 Technology stack

Table 5.1: Technology stack and rationale

Technology	Used for	Why
Python 3.12/3.13	Backend language	Mature async ecosystem, Pydantic and FastAPI; quick to test
FastAPI + Uvicorn	HTTP API, OpenAPI, dependency injection	Type-driven validation, async I/O, automatic schema/docs
Pydantic v2 / pydantic-settings	DTOs, request/response validation, configuration	Strict validation at trust boundaries; JSON schema
SQLAlchemy 2 (async) + aiosqlite	Persistence	Portable SQL; async sessions; SQLite for zero-setup laptops
Alembic	Schema migrations	Versioned DDL for non-development environments
httpx + tenacity	LLM provider HTTP clients and retries	One code path for five providers; jittered back-off
Ollama + Qwen2.5-Coder 7B (Q4_K_M)	Local LLM inference (optional)	No per-token cost; data stays local
passlib/bcrypt, hashlib.scrypt, HMAC-SHA256	Password hashing, JWT and capability tokens	Standard primitives; no external auth service
structlog	Structured logging with secret redaction	JSON logs for aggregation
prometheus-client	Metrics exposition	De-facto monitoring standard
Typer	Command-line interface	Typed CLI from Python functions
pytest, pytest-asyncio, pytest-cov	Backend testing and coverage	Fixtures for async code; coverage gate
Next.js 14 (App Router) + React 18 + TypeScript	Dashboard	File-based routing, server rewrites to the backend
TanStack Query 5, Zod 3	Data fetching/caching and runtime schema validation	Refetching, placeholders, typed parsing
Tailwind CSS 3.4, lucide-react, Recharts, sonner	Styling, icons, charts, toasts	Consistent design system quickly
Vitest 2 + Testing Library + jsdom	Frontend tests	Fast unit/component tests
Docker, Docker Compose	Container packaging (defined, not executed)	Reproducible deployment target
GitHub Actions	CI workflows	Automated lint/test/build
qdrant-client, redis-py	Optional vector store and cache clients	Future vector memory and caching; probed today

The backend's `pyproject` also declares `LangChain`, `LangGraph` and the official `OpenAI/Anthropic/Google` SDKs, but no module imports them: the provider adapters are implemented directly on `httpx`. We note this because it affects install size and because earlier project documents described `LangGraph`-based orchestration.

5.2 Code base overview

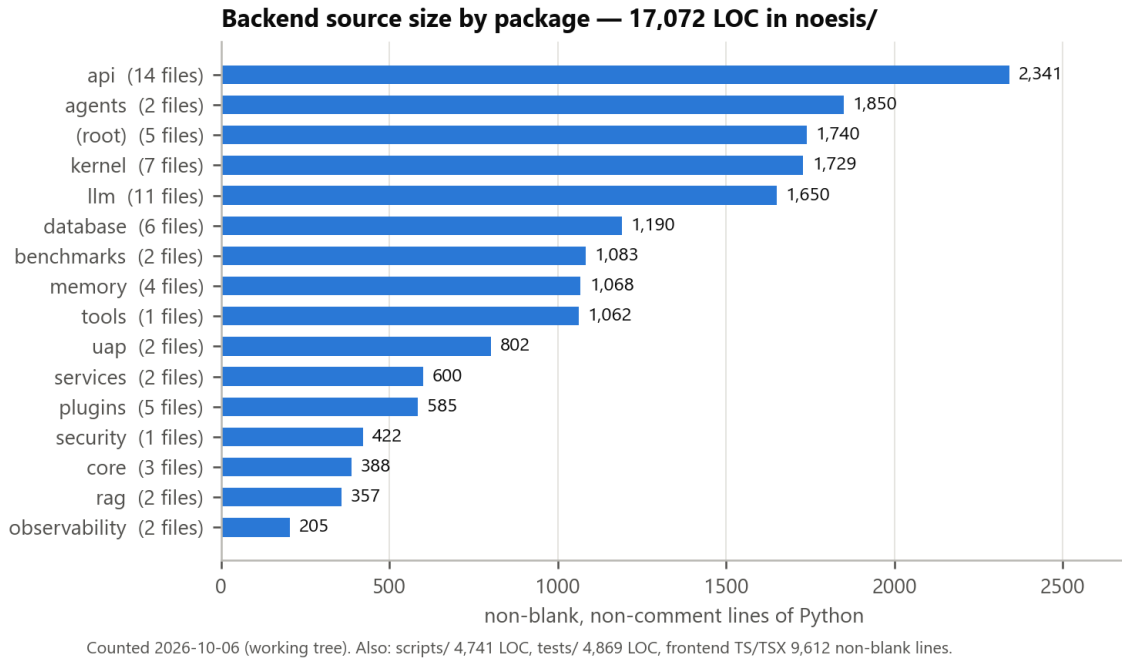


Figure 5.1: Backend source size by package (measured)

The backend package contains 17,072 non-blank, non-comment lines; the evaluation scripts 4,741; the tests 4,869; and the frontend 9,612 non-blank lines of TypeScript/TSX across 55 files.

5.3 Application bootstrap

`create_app()` builds the FastAPI instance, registers CORS (explicit origins), GZip, an idempotency-key echo and a request-id/access-log middleware that also feeds Prometheus. Four exception handlers wrap every error in the envelope. The `lifespan` builds the DI container, resolves and starts the kernel (scheduler loop and allocator GC), discovers plugins, creates tables in development, applies three in-place column additions and the tier table on SQLite, and warms Qdrant/Redis clients. Shutdown stops plugins and the container with two-second timeouts so tests on Windows do not hang.

5.4 Kernel and capabilities

`Capability.allows(op, target)` implements case-insensitive glob matching with prefix and suffix anchoring. `Kernel.syscall` rejects calls unless the kernel is running, records a denied span on capability failure, dispatches to the handler map and records the latency. The default handlers delegate memory reads to the SQL memory repository, account

writes in the allocator, call the registered chat port, route models, invoke registered tools and return the clock.

```
async def syscall(self, handle, call, payload=None):
    ctx = self._agents.get((handle.agent_id, handle.run_id))
    cap_op, target = self._syscall_to_capability(call, payload or {})
    if not any(c.allows(cap_op, target) for c in self._cap_table[(handle.agent_id,
handle.run_id)]):
        res = SysCallResult(call, success=False, error=str(PermissionDenied(...)))
        self._record_span(res, denied=True); return res
    result = await self._handlers[call](ctx, payload or {}) # 'No handler' if absent
    self._record_span(result); return result # excerpt, simplified
```

5.5 Agents and services

`PlannerSvc.plan` mints a planner token and runs `PlannerAgent`, which also drives a `MemoryAgent` that writes the plan and each step into tier T1 and runs three promotion passes. `AgentSvc.run_plan` orders steps with Kahn's algorithm, mints a per-step token (agent capabilities $\cup$ tool hints, TTL 900 s), shapes the state for coding, research and reflection agents, and calls `run_with_llm_or_fallback`. Because agents are synchronous Python, the HTTP layer off-loads the loop with `asyncio.to_thread`; each step creates a private event loop for the LLM call and the repository write.

Table 5.2: Agent implementations

Agent	Implementation technique	Notable limits
Planner	Keyword classification, mulberry32 RNG, UUIDv5 identities	English keyword tables; step text from templates
Research	web fetch over seed URLs; sentence scoring by query terms; opposing-keyword conflict detection	No seed URLs in plan runs $\Rightarrow$ AWAITING INPUT
Coding	Regex instruction parser; exact-match patching; difflib diff; SHA-256; writes new files	Only 'replace X with Y' style edits
Reflection	Citation, exit-code and contradiction heuristics	Pattern-based contradiction check
Judge / Critic	Lexical density, citation markers; five text-statistics axes	No semantic judgement
Executor	Severity-weighted criteria minus plan penalties; thresholds 0.90/0.50	Criteria synthesised when absent
Memory / RAG / Tool / Supervisor / Orchestrator	Summaries, synthetic chunks, batched tool calls, hash-derived task trees, worker lists	No real retrieval/dispatch

5.6 Memory subsystem

`MemoryTier` is a seven-method port (put, get, delete, search, score, summarize, prune) with in-memory adapters. Entry scores combine importance, a one-day recency half-life and access counts. The `PromotionController` holds T1–T6 tiers, thresholds, a Gyān fingerprint corpus, plan references and human-review marks; `run_all_promotions(passes)` evaluates edges bottom-up and appends to the hash chain. T6 refuses overwrites. `save_to_sql/load_from_sql` exist but no controller instance is registered for the lifespan hooks.

5.7 Tools and sandboxing

The registry checks `TOOL_INVOKE` on the tool name and normalises results. The default registry contains the shell tool in dry-run mode (it only describes the command unless an allow-list is configured), the Python sandbox (AST node allow-list, eighteen importable modules, `open/exec/eval/__import__` removed, execution in a daemon thread joined for five seconds), the files tool rooted at `var/workspace` with `resolve()`-based traversal checks, and an http/https-only fetcher that reuses the HTML parser.

5.8 LLM integration

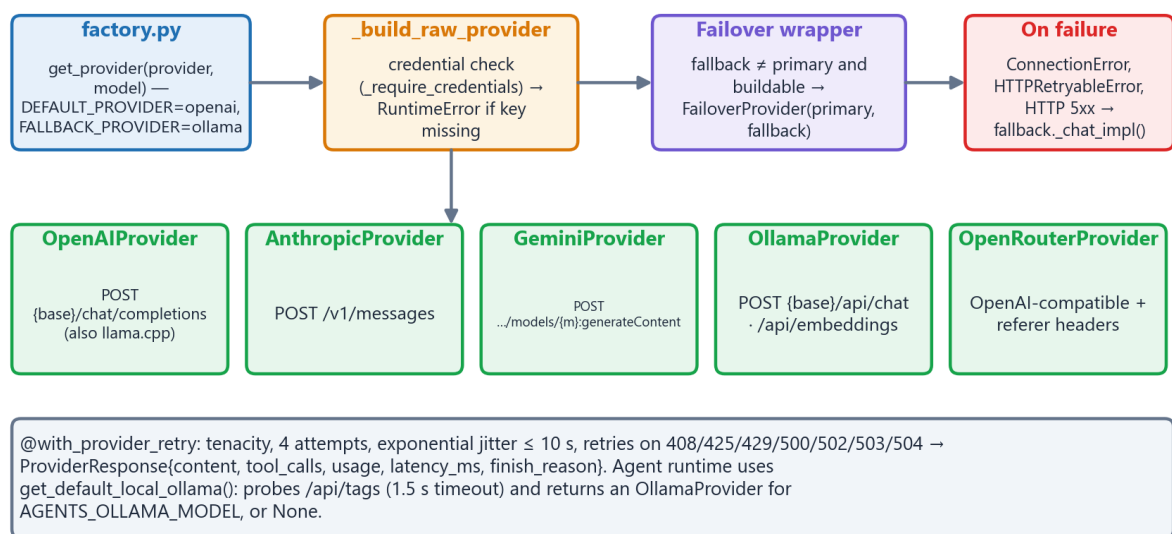


Figure 5.2: LLM provider factory, failover and adapters

5.9 Security implementation

JWTs are produced and verified by an in-house HS256 implementation (header/claims checks, constant-time comparison). Passwords use bcrypt via passlib, falling back to scrypt for inputs above bcrypt's 72-byte limit. The login route rate-limits per client and user, runs a dummy verification for unknown users, and returns uniform errors. The capability gate verifies `b64url(payload).b64url(HMAC)` tokens with Pydantic schema validation, expiry, required-capability and deny-mask checks.

5.10 Observability

Each request receives an X-Request-ID and one structured log line; secrets are redacted recursively. Prometheus counters and summaries cover HTTP requests; step and token metrics are defined but not yet called from the runtime. Kernel spans are kept in a 10 000-entry buffer and exposed through `/v1/kernel/traces`.

5.11 Frontend

The dashboard uses the Next.js App Router with a shared `DashboardShell`. `requestJson` adds headers and raises `HttpError` with toasts; `endpoints.ts` maps backend payloads to view models; `bench.ts` validates benchmark payloads with Zod and labels demo data. React Query provides caching and placeholder data. The conversation pages unwrap the API envelope and are live; other list helpers test for bare arrays and therefore fall back to seeded mocks.

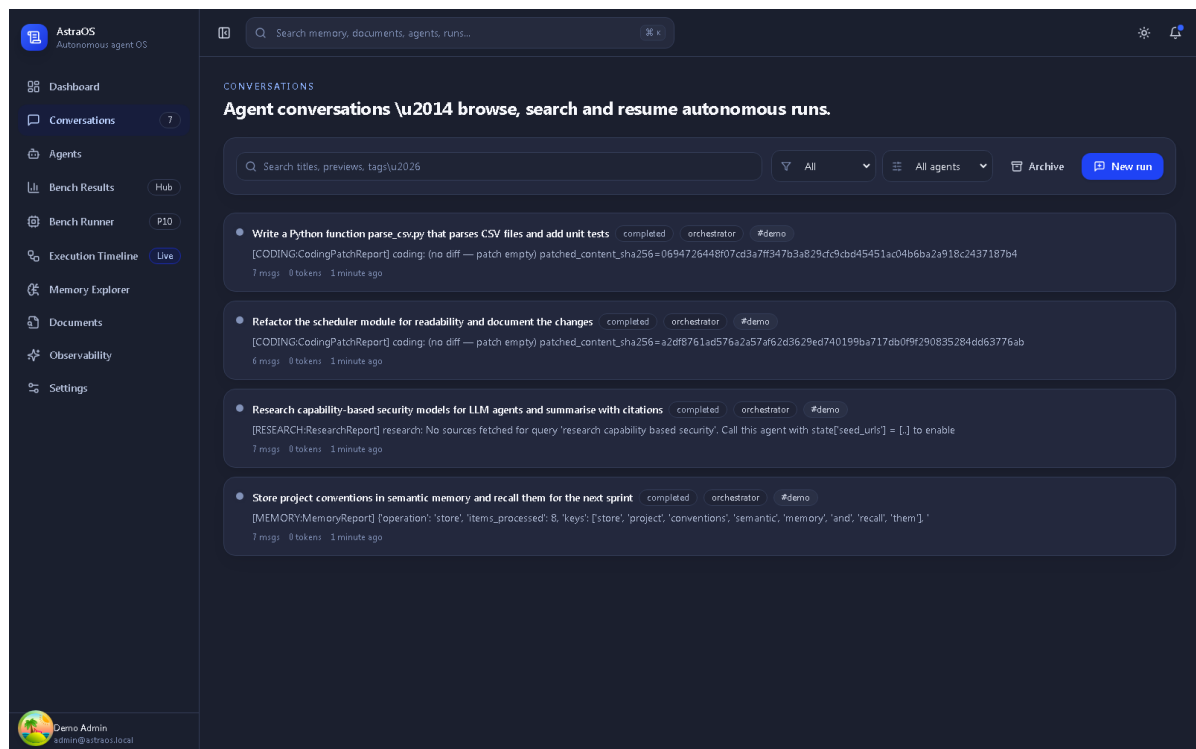


Figure 5.3: Conversations page of the running dashboard (live data)

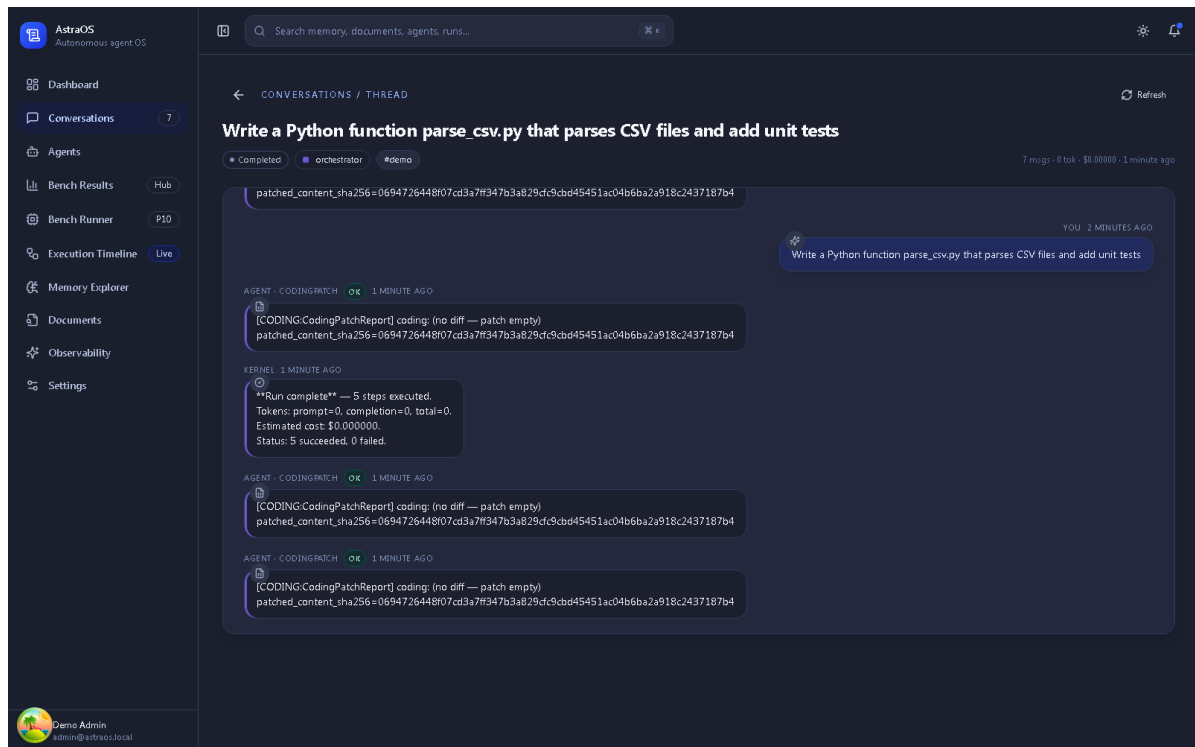


Figure 5.4: Thread view showing per-step agent reports (live data)

5.12 Evaluation harness

`bench_se50_batch.py` runs the seeded 12-phase pipeline per corpus goal and records the executor decision; `bench_humaneval.py` and `bench_mbpp.py` generate samples with Ollama in adaptive mode and check them with `compile()` in a subprocess; `run_w7_weekend.py` orchestrates all three; `benchmarks/stats.py` aggregates CSVs into `tables_for_paper.json` with bootstrap and Wilson intervals; `determinism_manifest.py` hashes repeated runs.

5.13 Deployment artefacts

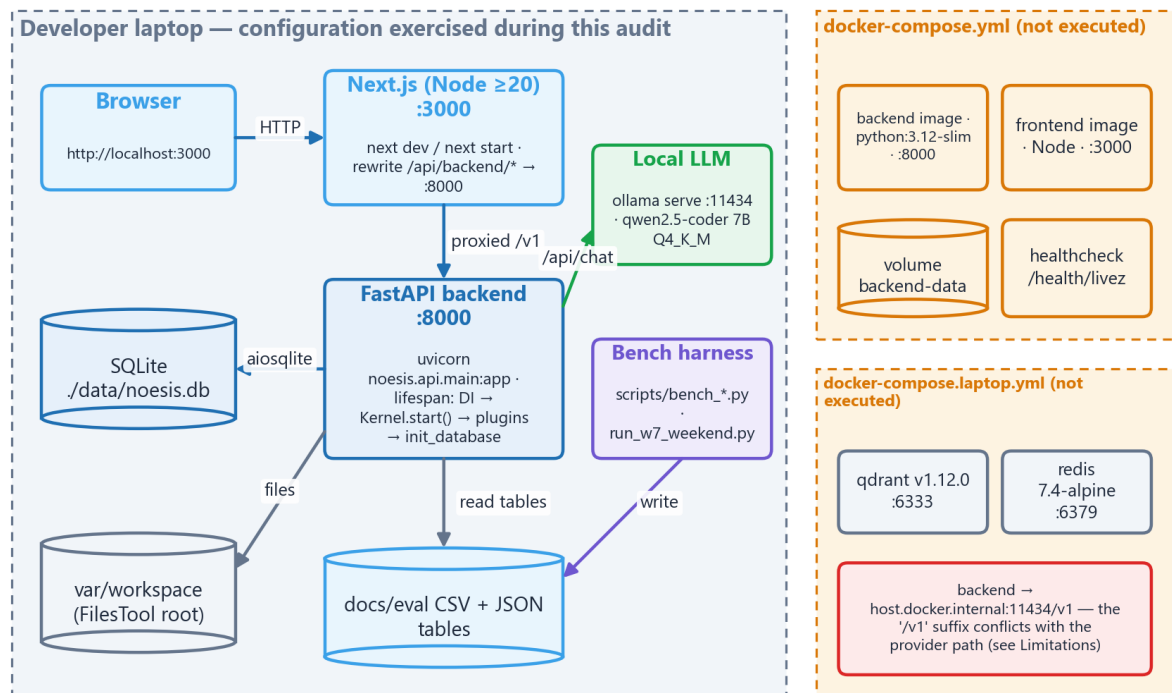


Figure 5.5: Deployment configuration

CHAPTER 6

METHODOLOGY

6.1 Development methodology

Development followed an incremental, milestone-based process. The code and changelog identify milestones: M0 (foundation: package, types, settings, ORM, providers, health probes, CI), M2 (memory tiers, memory bus, promotion pipeline, RAG library), M3 (tool registry, research/reflection/coding agents, security services, auth and observability routes), M5 (planner/agent services and the dashboard-parity v1 routes), M7 (the full twelve-agent roster) and W7 (evaluation runs with a local model). Each milestone added a test module (for example `test_m3.py`, `test_m5.py`) so that earlier behaviour remained protected. The public repository history begins on 5 October 2026 with an import commit, so per-milestone dates cannot be verified from git.

6.2 Architecture methodology

- **Map OS concepts to agent concepts** (process → agent, syscall → tool/memory/model operation, capability list → permission set, scheduler → step ordering, interrupt → asynchronous event).
- **Define types first:** shared Pydantic DTOs and ABC ports, then adapters.
- **Keep the kernel small and testable:** no I/O adapters inside `noesis.kernel`.
- **Prefer deterministic implementations** with optional LLM enrichment, to keep tests stable and runs reproducible.

6.3 Testing methodology

Unit tests target pure functions and classes (types, capabilities, planner, executor decision, memory tiers, RAG, UAP codecs, statistics). API tests use FastAPI's `TestClient` against an in-memory or temporary SQLite database. Claim suites assert allow/deny behaviour of the capability gate. The frontend uses Vitest with `jsdom` for helper, schema and component tests. Coverage is gated at 40 % in pyproject.

6.4 Evaluation methodology

Table 6.1: Evaluation questions and methods

Question	Method	Metric
Does the software behave as specified?	Run backend and frontend suites on an isolated copy	Pass/fail counts, coverage
Are permissions enforced?	Kernel, roster, tool-registry and HTTP-gate scenarios (allow and deny)	Expected vs observed outcome
Is planning reproducible?	Plan each SE-50 goal three times with seed 42; vary seeds	Unique SHA-256 per goal

Question	Method	Metric
How fast is it?	In-process latency with TestClient (n = 4–200 per endpoint); /llm/benchmark with Ollama	p50/p95 ms; tokens/s
Does memory promotion work and stay verifiable?	Controlled scenario with clock control, repeated five times	Tier counts; chain tail equality
How does retrieval perform?	Ingest three project documents; 50 queries each	Chunks, ingest ms, query μ s
What does the code harness show?	Re-analyse the committed W7 run (6 Oct 2026)	Compile-pass rate (unbiased), sign-off rate
What does the UI show?	Headless-browser screenshots of all routes against a seeded backend	Live vs placeholder data

6.5 Reproducibility controls

- The repository was not modified; every execution used copies in a temporary directory with a fresh SQLite database.
- Environment: Windows 11 Home (10.0.26200), Python 3.13.2, Node.js 20.18.1, FastAPI 0.139.2, Pydantic 2.13.4, SQLAlchemy 2.0.51, Ollama with qwen2.5-coder:7b-instruct-q4_K_M (7.6 B parameters, Q4_K_M, 4.68 GB).
- Deterministic mode was forced by pointing OLLAMA_BASE_URL to an unreachable port; LLM mode used the running Ollama server.
- All raw outputs (OpenAPI dump, test reports, coverage JSON, evidence JSON, screenshots) are shipped in `final_documentation/evidence`.
- Every figure caption states whether its data is measured, derived or reported.

6.6 Deployment methodology

The verified deployment is a two-process laptop setup (Uvicorn and Next.js). Container definitions and CI workflows exist and were reviewed statically; they were not executed during this work, and Chapter 8 lists the configuration defects that review found.

CHAPTER 7

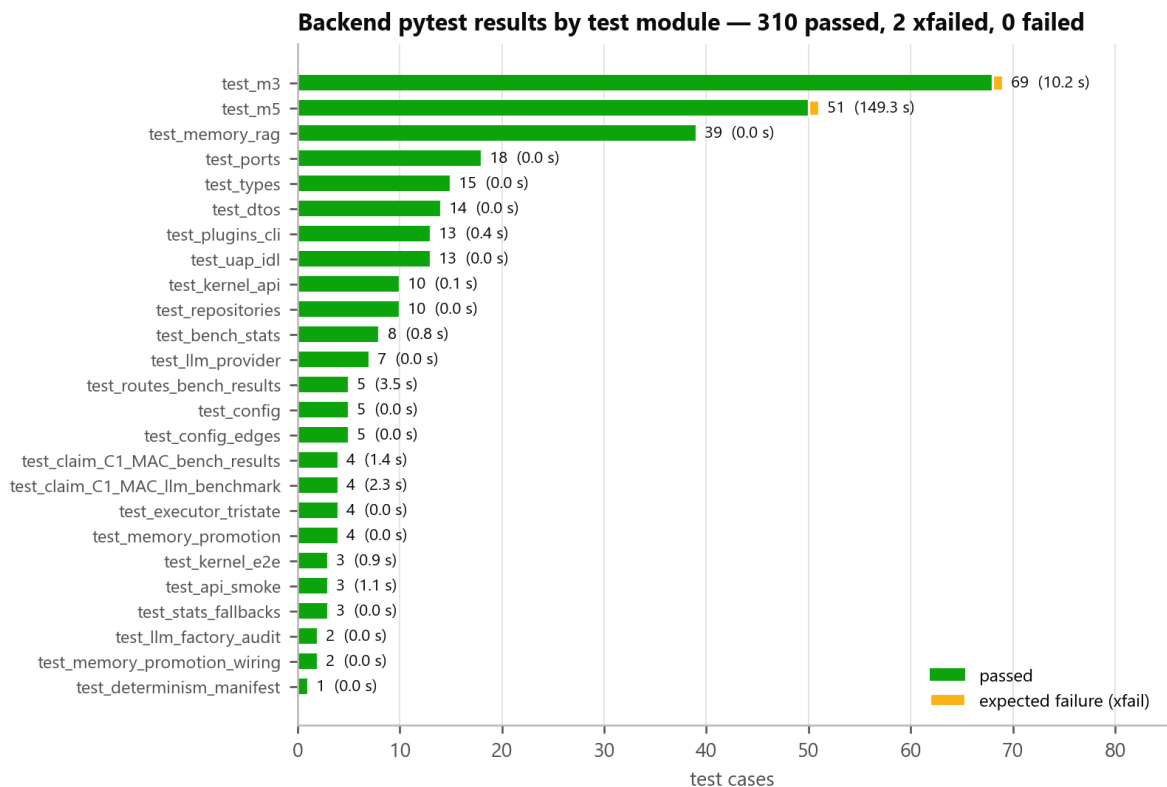
RESULTS AND EVALUATION

How to read this chapter: **Measured** results were obtained by us during this work on the stated environment. **Derived** results are computed from code constants. **Reported** results are re-analysed from artefacts committed to the repository. **Conceptual** statements are explanations, not data. Each table and figure carries one of these labels.

7.1 Functional testing

Table 7.1: Test results (measured)

Suite	Collected	Passed	Expected failures	Failed	Duration
Backend pytest (Python 3.13.2)	312	310	2	0	180.2 s
Frontend Vitest (Node 20.18.1)	105	103	0	2	—
TypeScript type check (tsc)	—	0 errors	—	—	—
ESLint (61 files, zero-warning gate)	—	0 errors / 0 warnings	—	—	—



Measured 2026-10-06, Python 3.13.2, Windows 11, isolated copy of the working tree; total wall time 180.2 s (test_m5 dominates because AgentSvc tests call the local LLM).

Figure 7.1: Backend test results per module (measured)

Every backend module passes. The two expected failures document deliberate deviations: the observability summary is not admin-gated in this build, and the memory agent is now registered so a test expecting `AWAITING_INPUT` no longer applies. The long duration

of `test_m5` (149 s) comes from agent-runtime tests that call the local LLM when it is available.

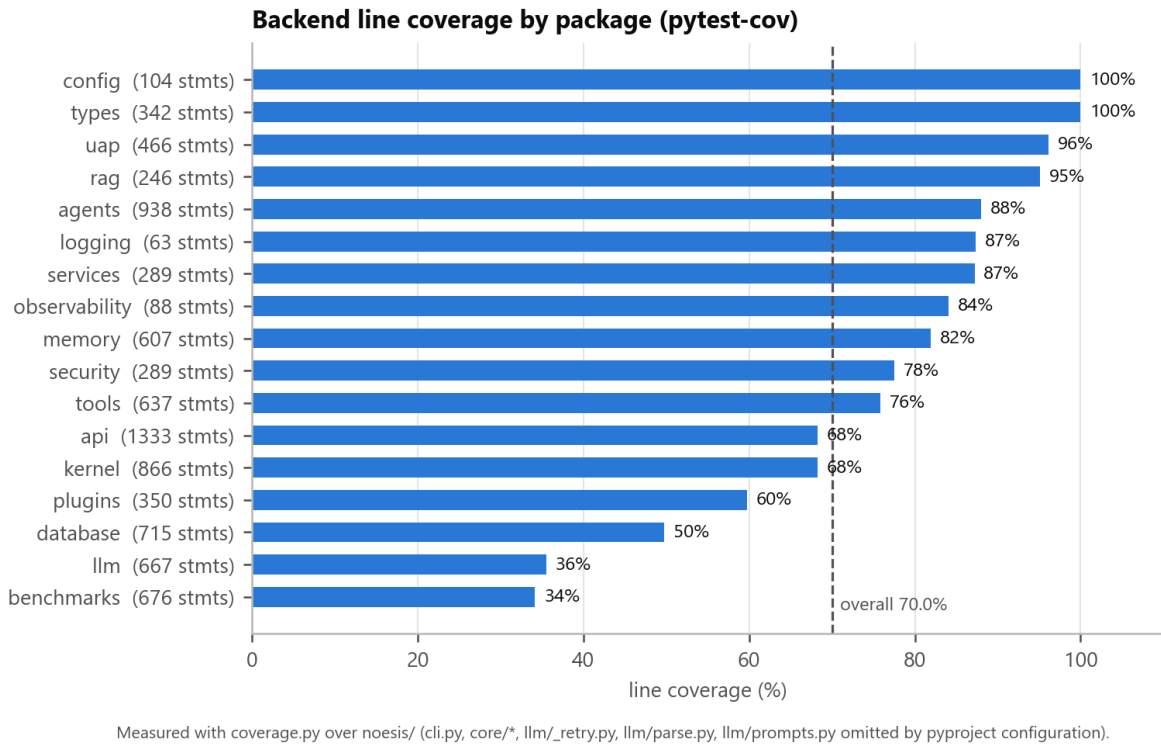


Figure 7.2: Line coverage per package (measured)

Overall 70.0 % of 8,683 statements.

Two frontend tests fail on the working tree: both assert that `API_BASE` defaults to `/api/backend/v1`, while the uncommitted version of `http.ts` uses `/api/backend` (endpoint paths now include `/v1`). The code change is consistent with the endpoints; the tests were not updated.

7.2 Security and permission enforcement

Table 7.2: Kernel syscall capability scenarios (measured)

Kernel scenario	Outcome	Detail
K1 zero capabilities -> TOOL_INVOKE(search)	denied	PermDenied[token=201d6308-f538-4ed0-8077-5aef6d6d4402 op=tool_invoke t
K2 TOOL_INVOKE:search -> TOOL_INVOKE(shell)	denied	PermDenied[token=d1096f53-d486-476b-895a-54c5d9540065 op=tool_invoke t
K3 TOOL_INVOKE:search -> TOOL_INVOKE(search)	allowed	—
K4 TOOL_INVOKE:s* glob -> TOOL_INVOKE(shell)	allowed	—
K5 MEMORY_READ:* -> MEMORY_WRITE(working)	denied	PermDenied[token=866c68d4-9a96-4700-81e1-dcf8bbabce10 op=memory_write
K6 MEMORY_WRITE:working -> MEMORY_WRITE(working)	allowed	—
K7 MODEL_INFERENCE:* -> MODEL_ROUTE	allowed	—

Kernel scenario	Outcome	Detail
K8 MODEL_INFERENCE:* -> SPAWN_AGENT	denied	PermDenied[token=4683f779-737f-4b22-aaca-7b54cd55ff9c op=spawn_agent t
K9 all caps -> RAG_SEARCH (no handler)	denied	No handler for syscall rag_search

Table 7.3: Capability-gate scenarios on HTTP routes (measured)

HTTP gate scenario	Status	Response error
llm benchmark no token	403	C1_DENY_MISSING_CAP: missing X-Noesis-Capability-Token header (MAC-gated endpoint requires a signed capability token)
llm benchmark malformed token	403	C1_DENY_MAC: malformed capability token (expected b64payload.b64mac)
llm benchmark wrong secret	403	C1_DENY_MAC: capability token HMAC signature verification failed
llm benchmark expired	403	C1_DENY_EXPIRED: capability token expired at unix ts=1791296748
llm benchmark missing cap	403	C1_DENY_MISSING_CAP: caller lacks required capability 'llm.benchmark.read' (capabilities on token: bench.results.read)
llm benchmark deny mask	403	C1_DENY_MASK: deny mask 'llm.*' matched required capability 'llm.benchmark.read' (negative capability precedence: deny always wins)
llm benchmark ok	200	ok
bench results ok	200	ok
bench results no token	403	C1_DENY_MISSING_CAP: missing X-Noesis-Capability-Token header (MAC-gated endpoint requires a signed capability token)

The roster refused to spawn each of the 12 agent types with an empty capability set and allowed each one with exactly its declared capabilities (12/12). The tool registry denied the files tool to a caller holding only TOOL_INVOKE:web_fetch, refused a path-traversal read with exit code 13, described but did not execute `rm -rf /`, and blocked `import os` and `open()` in the Python sandbox (measured).

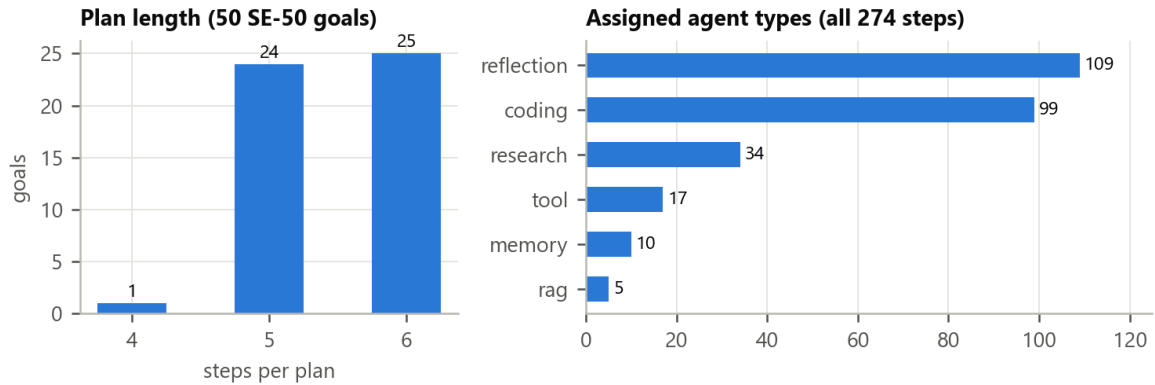
Table 7.4: Authentication scenarios (measured)

Authentication scenario	Status	Result
login ok	200	token pair / claims returned
me ok	200	token pair / claims returned
me missing token	401	missing bearer token
me tampered token	401	jwt signature mismatch
me with refresh token (wrong use)	401	jwt token_use must be 'access'
refresh ok	200	token pair / claims returned
login bad password	401	invalid credentials
login unknown user	401	invalid credentials
login validation error (short pw)	422	Request validation failed
login rate limited (7th)	429	HTTP 429

Login attempts 1–5 with a wrong password returned 401 and the sixth returned 429 with `retry_after_s = 60` (measured: [401, 401, 401, 401, 401, 429]). **Negative finding:** the resource routes accepted unauthenticated requests, e.g. `GET /v1/conversations` returned 200 without any token.

7.3 Determinism and reproducibility

PlannerAgent on SE-50: 50/50 goals hash-identical over 3 runs · median plan time 1.05 ms



Measured 2026-10-06 via PlannerSvc.plan(goal, seed=42) on backend/benchmarks/noesis_se50/corpus.csv.

Figure 7.3: Planner behaviour on the SE-50 corpus (measured)

All 50 goals produced a single SHA-256 over three runs; plans have 4–6 steps; reflection and coding dominate the agent mix because the keyword table assigns 'file', 'test', 'build' and similar words to coding and the final step is always reflection. Five different seeds produced five distinct plans for the same goal, confirming that determinism is seed-controlled rather than constant.

Table 7.5: Reproducibility results

Experiment	Result	Type
Planner: 50 SE-50 goals × 3 runs (seed 42)	50/50 single hash	Measured
Planner timing (150 plans)	median 1.05 ms, p95 1.64 ms	Measured
Seed sensitivity (5 seeds)	5 distinct hashes	Measured
Promotion chain, controlled scenario × 5	1 distinct tail hash	Measured
Determinism manifest (docs/eval, 50 goals × 3)	450/450 ordered identical pairs (= 50/50 groups); 141/150 runs with pipeline status FAILED	Reported
Same plan with and without LLM	Identical step status/report; only preview text and tokens differ	Measured

7.4 Performance

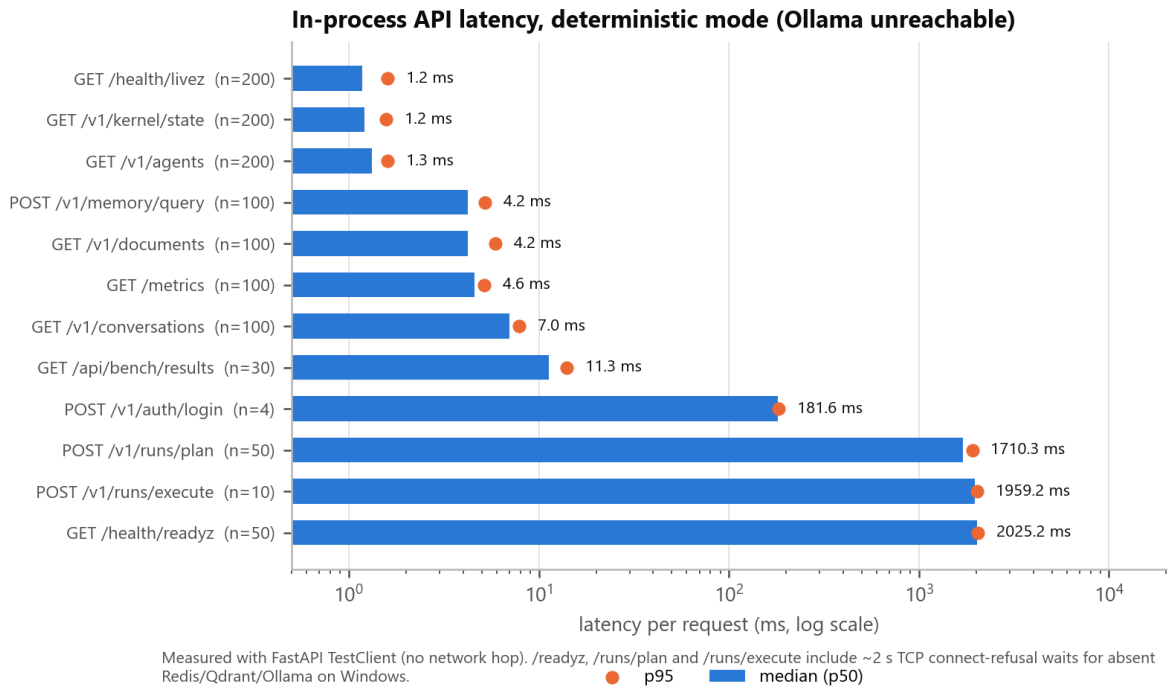


Figure 7.4: In-process API latency (measured, deterministic mode)

Table 7.6: API latency (measured, in-process, deterministic mode)

Endpoint	n	Mean (ms)	p50 (ms)	p95 (ms)	Max (ms)
GET /health/livez	200	1.2	1.2	1.6	2.6
GET /v1/kernel/state	200	1.3	1.2	1.6	2.4
GET /v1/agents	200	1.3	1.3	1.6	1.9
POST /v1/memory/query	100	4.3	4.2	5.2	6.0
GET /v1/documents	100	4.5	4.2	5.9	7.7
GET /metrics	100	4.6	4.6	5.2	5.8
GET /v1/conversations	100	7.1	7.0	7.9	8.7
GET /api/bench/results	30	9.9	11.3	14.0	15.4
POST /v1/auth/login	4	180.5	181.6	182.6	182.6
POST /v1/runs/plan	50	1755.6	1710.3	1910.0	1945.5
POST /v1/runs/execute	10	1949.1	1959.2	2025.3	2025.3
GET /health/readyz	50	2025.4	2025.2	2037.0	2043.5

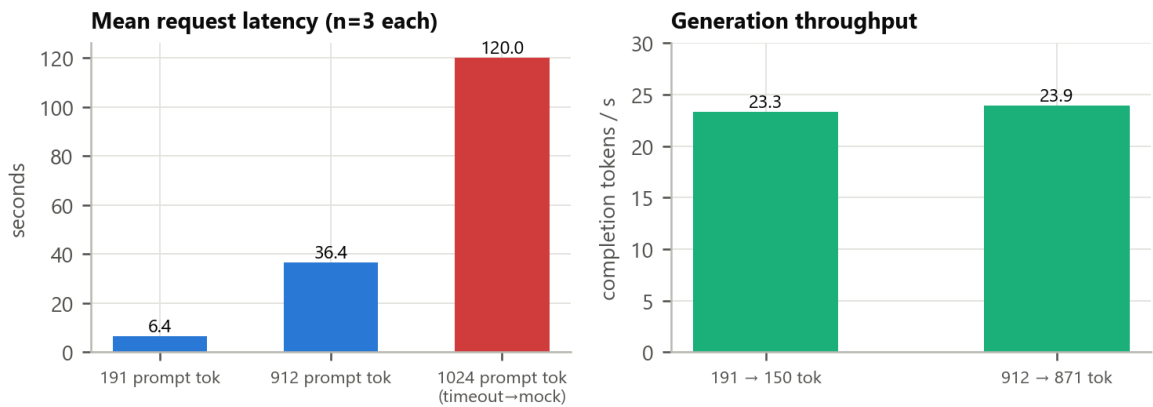
Reading the table: endpoints that only touch memory or a small SQLite query answer in about 1–11 ms. Login is dominated by deliberate password-hashing cost. Plan and execute requests are dominated by the Ollama availability probe made while constructing services for each request (≈ 2 s here because the port was closed on Windows); the planner itself needs about 1 ms (Table 7.5). With Ollama running, the same plan request took 0.64 s.

Table 7.7: GET /llm/benchmark runs against qwen2.5-coder:7b-instruct-q4_K_M (measured)

Prompt tokens	Completion tokens	Latency (s)	Completion tok/s	Mock fallback
---------------	-------------------	-------------	------------------	---------------

Prompt tokens	Completion tokens	Latency (s)	Completion tok/s	Mock fallback
191	150	6.7	22.5	no
191	150	6.3	23.9	no
191	150	6.4	23.5	no
912	871	37.6	23.1	no
912	871	35.9	24.2	no
912	871	35.8	24.3	no
1024	128	120.0	1.1	yes
1024	128	120.0	1.1	yes
1024	128	120.0	1.1	yes

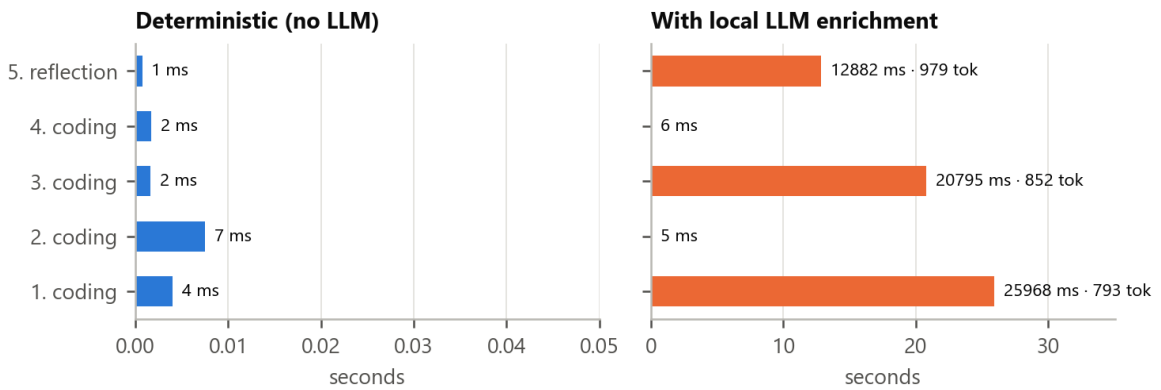
GET /llm/benchmark against local Ollama (qwen2.5-coder:7b-instruct-q4_K_M)



Measured 2026-10-06 on the author's laptop (CPU/GPU not profiled). The 1024-token requests exceeded the 120 s wait_for_timeout and the endpoint returned mock values.

Figure 7.5: Local-LLM latency and throughput through /llm/benchmark (measured)

POST /v1/runs/execute — per-step duration for the same 5-step plan



Measured 2026-10-06. Wall time 1.8 s (deterministic, incl. provider probe) vs 60.4 s (LLM). Two LLM-mode steps returned with zero tokens; the cause is not logged.

Figure 7.6: Per-step duration of the same five-step plan without and with LLM enrichment (measured)

Without an LLM each step takes 1–7 ms. With qwen2.5-coder 7B the three steps that reached the model took 13–26 s and consumed 793–979 tokens; two steps returned in about 5 ms with no tokens, a behaviour whose cause the logs do not record. The end-to-

end request took 60.4 s, and the conversation-level background run 49.5 s with 2 594 tokens.

7.5 Memory promotion

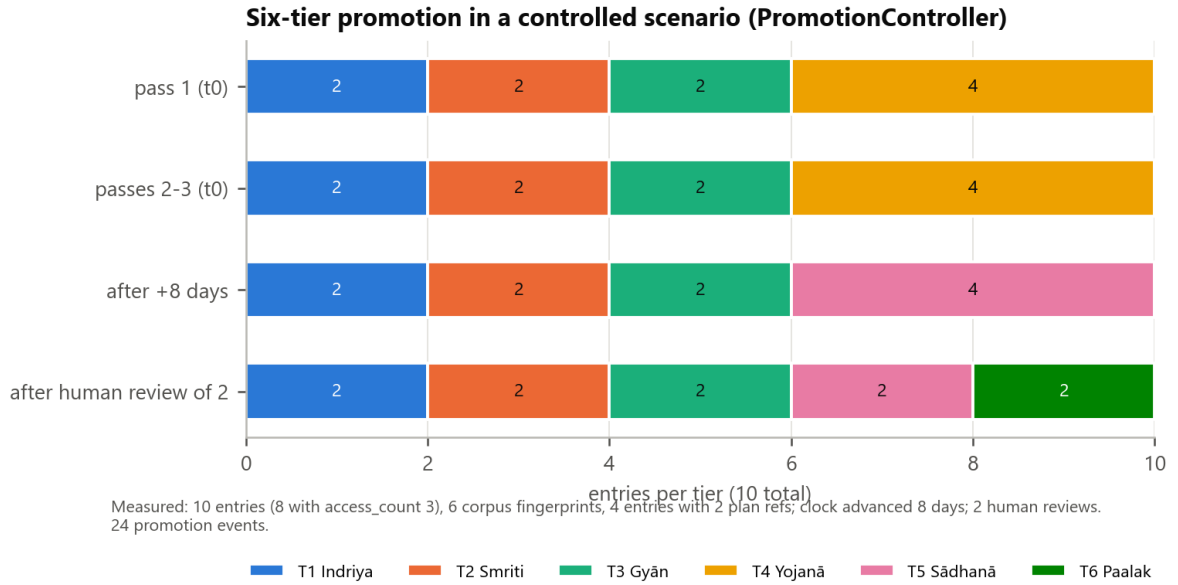


Figure 7.7: Tier occupancy in a controlled promotion scenario (measured)

The controlled scenario exercised all five edges (events per edge: {'T1->T2': 8, 'T2->T3': 6, 'T3->T4': 4, 'T4->T5': 4, 'T5->T6': 2}), the T6 tier refused an overwrite (refused: Paalak tier refuses overwrite of 92dc97a3-fe78-4bb7...), and the chain tail was identical across five repetitions. In live planning, however, the planner's promotion report showed T1 = 5 and all other tiers 0 with an all-zero provenance tail, because step entries are created with an access count of 1 and no corpus or plan references reach the thresholds. The promotion model is therefore verified as a component but not yet exercised by real workloads.

7.6 Retrieval and benchmark harness

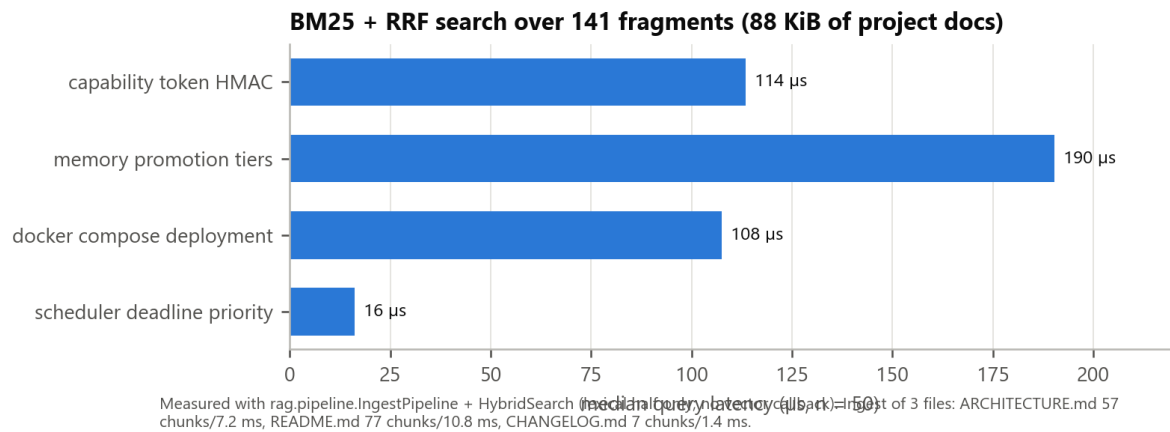
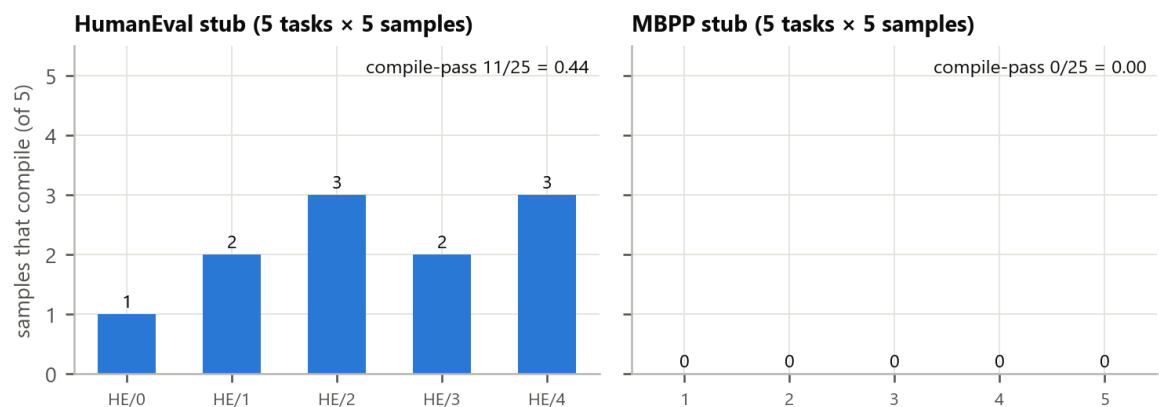


Figure 7.8: BM25 + RRF query latency over project documents (measured)

Table 7.8: Benchmark results (reported, re-analysed)

Benchmark (W7 run, 2026-10-06)	Setting	Result	Interpretation
SE-50 (5 tasks × 3 runs)	Seeded pipeline; sign-off criteria synthesised as met	15/15 SIGNOFF	Pipeline completes without error; not task correctness
HumanEval stub (5 × 5)	qwen2.5-coder 7B, T 0.7; compile() only	11/25 compile (0.44)	Syntactic validity only
MBPP stub (5 × 5)	Same	0/25 compile	Generated samples did not compile
Hosted models (GPT-4o etc.)	—	No artefacts	README figures not reproducible

Syntax-compile pass per task — W7 run of 2026-10-06 (qwen2.5-coder 7B, T = 0.7)



Source: docs/eval/w7_weekend_real_Oct06_real. The harness checks compile() only — it does not execute unit tests, so these are NOT functional pass@k scores.

Figure 7.9: Compile-pass per task in the W7 run (reported)

7.7 User-interface evaluation

All 13 routes compiled and rendered (HTTP 200) in the development server. With a seeded backend, the conversation list, the new-run form and the thread view displayed live data; the other monitoring pages displayed seeded placeholder data because their

helpers expect bare arrays while the API returns envelopes. The benchmark pages label their demo data explicitly.

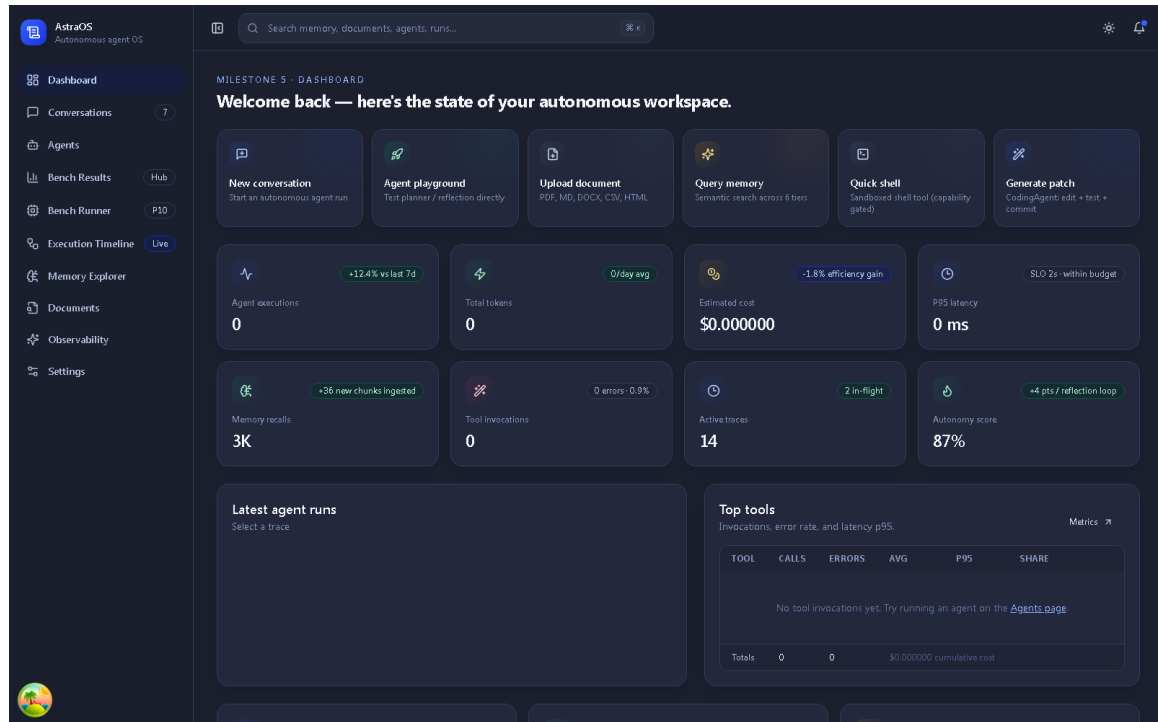


Figure 7.10: Dashboard (placeholder metrics) — screenshot of the running application

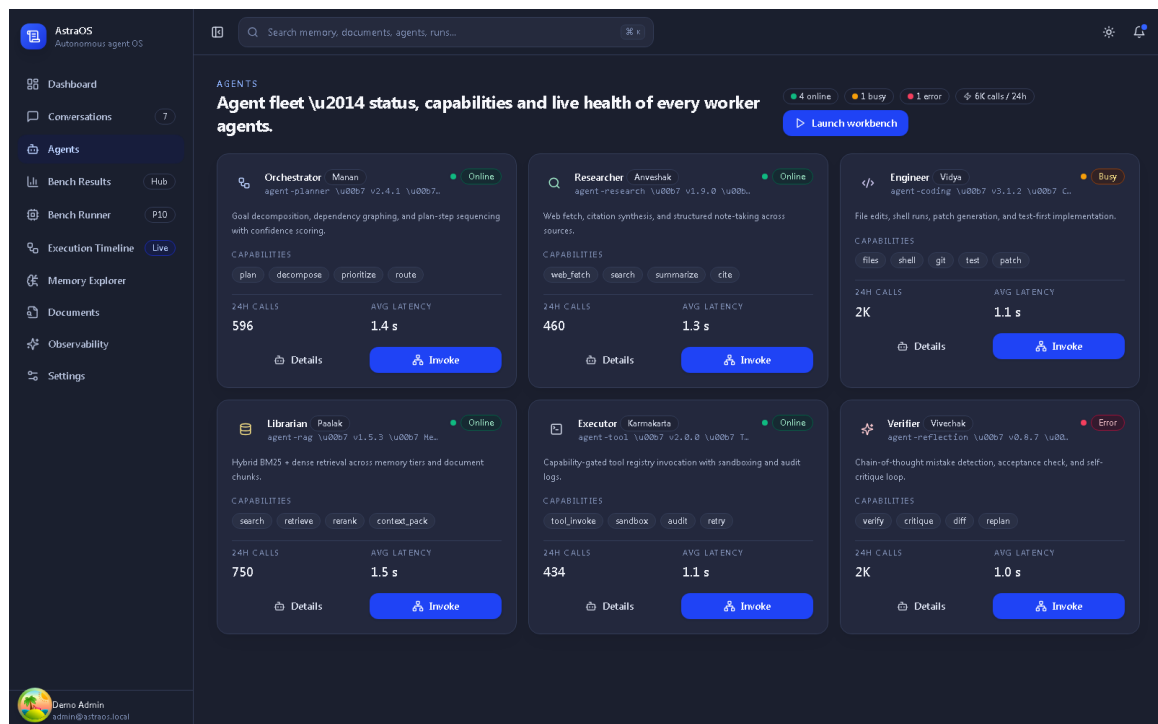


Figure 7.11: Agents page (placeholder fleet) — screenshot of the running application

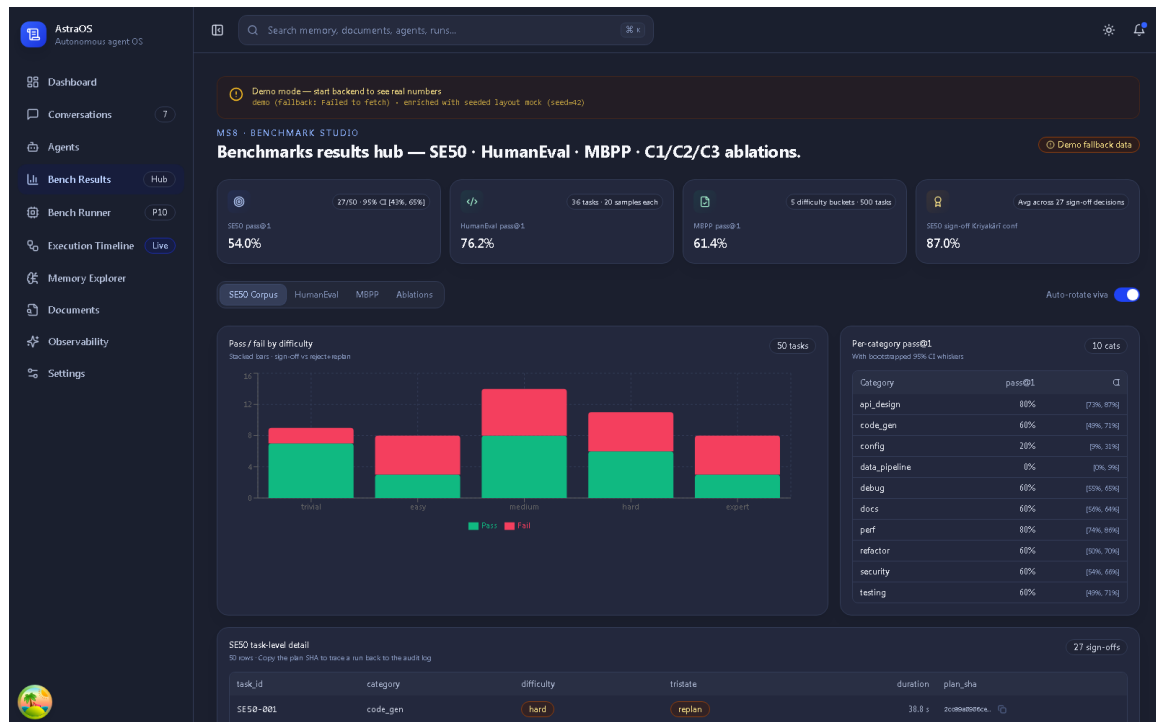


Figure 7.12: Bench Results Hub with labelled demo data — screenshot of the running application

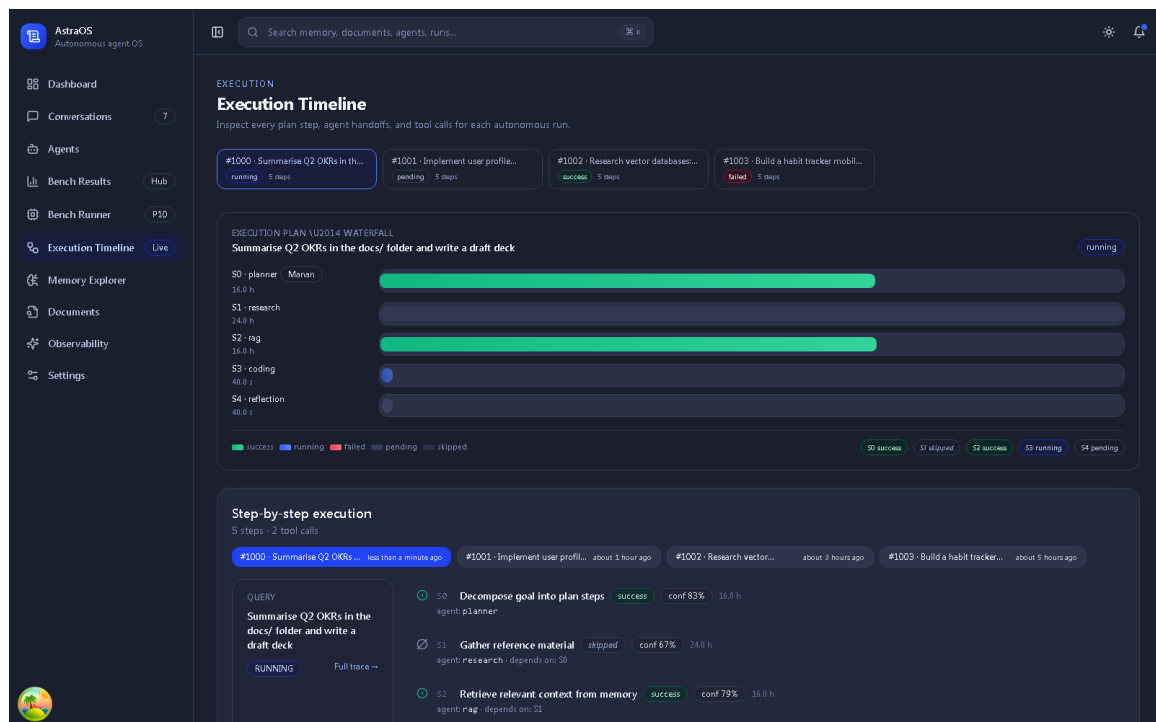


Figure 7.13: Execution timeline (placeholder traces) — screenshot of the running application

7.8 Attainment of objectives

Table 7.9: Achievement of objectives

Objective	Evidence	Assessment
O1 Kernel with typed agents, capabilities, tracing	Kernel tests; 9 kernel scenarios; 12-agent roster	Achieved
O2 Deterministic planning	50/50 goals, seed sensitivity, ~1 ms per plan	Achieved

Objective	Evidence	Assessment
O3 Least-privilege enforcement	Roster/tool/kernel/HTTP-gate scenarios	Achieved for agent paths; not for resource HTTP routes
O4 Tiered memory with provenance	Controlled scenario; T6 immutability; chain reproducibility	Achieved as a component; not exercised in live runs
O5 Optional LLMs and characterisation	Fallback measured; 23–24 tok/s; timeout behaviour	Achieved
O6 API and dashboard with tests	42 operations; 310/312 + 103/105 tests; screenshots	Partially achieved (6 pages show placeholders)
O7 Evaluation harness with statistics	Harness scripts, stats module, W7 artefacts	Partially achieved (compile-only, small run)

7.9 Comparative evaluation

A quantitative head-to-head comparison with other frameworks was not performed: no comparable runs exist in the repository, and comparing a rule-based agent pipeline with LLM-driven frameworks on task success would not be meaningful. Table 2.1 in Chapter 2 provides the qualitative comparison of system-level concerns; the measured results above support the NOESIS column for permissions, determinism and LLM-optional operation, and qualify it for memory and operations.

CHAPTER 8

DISCUSSION

8.1 What worked and why

- **Determinism through identity design.** Deriving step and plan identities with UUIDv5 from a seed string that includes a SHA-256 of the goal — instead of Python's randomised `hash()` or wall-clock time — made byte-identical plans a property of the code rather than of test fixtures.
- **Capabilities as data.** Declaring required capabilities on each agent class allowed three independent enforcement points (roster, tool registry, kernel) to use the same information, and made least-privilege tokens trivial to mint per step.
- **Separation of decision and generation.** Because the LLM never decides the structured output, an unavailable or slow model degrades the system to deterministic mode instead of breaking it.
- **Ports and Pydantic everywhere.** Strict DTOs at boundaries and injectable ports produced a test suite that runs without external services and reaches 70 % coverage.

8.2 Trade-offs

Table 8.1: Design trade-offs

Decision	Benefit	Cost
Rule-based agents	Reproducible, fast (ms), testable	Low task intelligence; LLM output unused for decisions
Background task in the request process	Simple; immediate response	Lost on restart; no queue or retry
In-process state (tiers, spans, limits)	No infrastructure	No horizontal scaling; no persistence
Per-request service construction	Picks up Ollama availability dynamically	~0.6–2 s probe overhead per plan/execute request
SQLite default	Zero setup	Single writer; SQLite-specific migrations
Custom JWT implementation	No extra dependency	Must be maintained and reviewed carefully

8.3 Challenges and unexpected findings

- **Envelope contract drift.** The backend standardised on `{ok, data, ...}` while several frontend helpers still expected arrays, silently turning live pages into mock pages.
- **Run identity.** Task executions store `run_id` only in JSON metadata, and the repository filters by conversation id; the run-executions endpoint therefore returns nothing for real run ids.
- **Windows connection refusal.** Probing absent services costs about two seconds per probe on Windows, which dominated plan/execute latency when Ollama was down.

- **Benchmark semantics.** The harness's `pass_at_1` fields reflect sign-off or compilation, not functional tests; re-analysis was required to report them correctly.
- **Documentation inflation.** Several README statements (test counts, route names, comparative benchmark numbers, LangGraph use, BCa intervals) did not match the code (SRS Appendix F).

8.4 Threats to validity

- Latency was measured in-process on one laptop; production numbers will differ.
- The W7 benchmark run used five-task stub corpora; it cannot support claims about model quality.
- LLM throughput depends on the unprofiled CPU/GPU of the test machine.
- Container deployment and CI execution were not verified.

8.5 Limitations

The principal limitations are rule-based agents, unauthenticated resource routes, simulated document ingestion and retrieval agent, unwired memory persistence and observability counters, six placeholder dashboard pages, compile-only code benchmarks and single-process state. The SRS (Chapter 20) lists them with evidence.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Summary

NOESIS applies operating-system ideas to a multi-agent LLM runtime. A goal becomes a deterministic plan; a runtime executes the plan with twelve typed agents under per-step, least-privilege capabilities; every step is audited; memory can be promoted through six tiers with a hash-chained provenance log; and an optional LLM contributes rationale without controlling decisions. The system is implemented as a FastAPI service with a Next.js dashboard and is covered by 312 backend and 105 frontend tests.

9.2 Contributions revisited

- A tested agent microkernel (syscalls, capability table, spans, scheduler, allocator, router, interrupts).
- Least-privilege execution measured across kernel, roster, tools and HTTP gate.
- Bit-identical planning measured on 50 goals, at about one millisecond per plan.
- A provenance-logged six-tier memory controller with reproducible hash chains.
- An LLM-optional runtime characterised with a local 7B model (≈ 23 tok/s).
- An evidence-based evaluation that reconciles project claims with the implementation.

9.3 Achievement of objectives

Objectives O1, O2 and O5 were achieved; O3 and O4 were achieved for the kernel paths and as a component respectively; O6 and O7 were partially achieved (Table 7.9).

9.4 Future work

1. Authenticate and authorise all resource routes and reject default secrets in production.
2. Replace heuristic agents with LLM-backed agents that emit the same typed reports through structured-output parsing, keeping deterministic fallbacks.
3. Connect the BM25/RRF library and an embedding store to document upload and the RAG agent.
4. Drive memory promotion from real recall statistics and register the controller for persistence.

5. Store `run_id` as a column and stream step progress to the dashboard; unwrap envelopes so all pages show live data.
6. Execute unit tests in HumanEval/MBPP (functional pass@k) and evaluate on SWE-bench Lite with paired statistics.
7. Externalise state for horizontal scaling; fix and verify container deployment; characterise edge hardware as proposed in the synopsis.

References

- [1] R. C. Atkinson and R. M. Shiffrin, "Human memory: A proposed system and its control processes," in *The Psychology of Learning and Motivation*, vol. 2, K. W. Spence and J. T. Spence, Eds. New York: Academic Press, 1968, pp. 89–195, doi: 10.1016/S0079-7421(08)60422-3.
- [2] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.
- [3] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.
- [4] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability myths demolished," Systems Research Laboratory, Johns Hopkins University, Tech. Rep. SRL2003-02, 2003.
- [5] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974, doi: 10.1145/361011.361061.
- [6] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962, doi: 10.1145/368996.369025.
- [7] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [8] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOS: LLM agent operating system," arXiv:2403.16971, 2024.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
- [10] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, arXiv:2201.11903.
- [11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.
- [12] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2302.04761.
- [13] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
- [14] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2308.00352.
- [15] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for 'mind' exploration of large language model society," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.17760.

- [16] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, 2023, arXiv:2304.03442.
- [17] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024, arXiv:2308.11432.
- [18] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023.
- [19] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, arXiv:2005.11401.
- [20] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/15000000019.
- [21] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
- [22] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.
- [23] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
- [24] J. Austin *et al.*, "Program synthesis with large language models," arXiv:2108.07732, 2021.
- [25] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2310.06770.
- [26] B. Hui *et al.*, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, arXiv:2302.12173.
- [28] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th ACM Symp. Theory of Computing (STOC)*, 2002, pp. 380–388, doi: 10.1145/509907.509965.
- [29] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.
- [30] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, 1997.
- [31] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.
- [32] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

- [33] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.
- [34] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [35] J. Pineau *et al.*, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [36] A. Cockburn, "Hexagonal architecture (ports and adapters)," 2005. [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture/>
- [37] "IEEE recommended practice for software requirements specifications," IEEE Std 830-1998, 1998.
- [38] "Systems and software engineering — Life cycle processes — Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.
- [39] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [40] S. Ramírez, "FastAPI," software framework. [Online]. Available: <https://fastapi.tiangolo.com/>
- [41] Pydantic Services, "Pydantic v2 documentation." [Online]. Available: <https://docs.pydantic.dev/>
- [42] SQLAlchemy authors, "SQLAlchemy 2.0 documentation." [Online]. Available: <https://docs.sqlalchemy.org/>
- [43] Vercel, "Next.js documentation (App Router)." [Online]. Available: <https://nextjs.org/docs>
- [44] Ollama, "Ollama API documentation." [Online]. Available: <https://github.com/ollama/ollama/blob/main/docs/api.md>
- [45] Prometheus authors, "Exposition formats." [Online]. Available: https://prometheus.io/docs/instrumenting/exposition_formats/
- [46] LangChain, "LangChain and LangGraph documentation." [Online]. Available: <https://python.langchain.com/>
- [47] CrewAI, "CrewAI documentation." [Online]. Available: <https://docs.crewai.com/>

Note: bibliographic details were compiled by the authors from the cited works and the project's bibliography; page ranges and DOIs should be checked against publisher records before final submission. The venue of [22] is given as TACL 2024 (the synopsis cited an ACL 2023 DOI for this work).

Appendix A: Test Inventory

Table A.1: Backend test modules (measured)

Test module	Passed	xfail	Seconds
api/test_routes_bench_results	5	0	3.55
claim_suites/test_claim_C1_MAC_bench_results/TestClaimC1MACBenchResults	4	0	1.41
claim_suites/test_claim_C1_MAC_llm_benchmark/TestClaimC1MACLLMBenchmark	4	0	2.35
integration/test_kernel_e2e	3	0	0.95
unit/test_api_smoke	3	0	1.06
unit/test_bench_stats	8	0	0.79
unit/test_config	5	0	0.03
unit/test_config_edges	5	0	0.01
unit/test_determinism_manifest	1	0	0.04
unit/test_dtos	14	0	0.01
unit/test_executor_tristate	4	0	0.01
unit/test_kernel_api	10	0	0.09
unit/test_llm_factory_audit	2	0	0.03
unit/test_llm_provider	7	0	0.03
unit/test_m3	68	1	10.24
unit/test_m5	50	1	149.27
unit/test_memory_promotion	4	0	0.04
unit/test_memory_promotion_wiring	2	0	0.01
unit/test_memory_rag	39	0	0.05
unit/test_plugins_cli	13	0	0.38
unit/test_ports	18	0	0.02
unit/test_repositories	10	0	0.02
unit/test_stats_fallbacks	3	0	0.02
unit/test_types	15	0	0.02
unit/test_uap_idl	13	0	0.03

Appendix B: Sample Plan

PlannerSvc output for the goal “Research capability-based security and write a summary document with citations” (seed 42), abridged:

```
{
  "id": "b449d0a6-7990-571e-8f3a-56b4a1169d1a",
  "goal": "Research capability-based security and write a summary document with citations",
  "created_at": "2026-01-01T00:00:00Z",
  "steps": [
    {
      "index": 0,
      "assigned_agent": "research",
      "description": "Gather authoritative sources on security",
      "dependencies": [],
      "tool_hints": [
        "web_fetch"
      ],
      "confidence": 0.9
    },
    {
      "index": 1,
      "assigned_agent": "research",
      "description": "Cross-check claims about research across ≥2 references",
      "dependencies": [
        "bee9408d..."
      ],
      "tool_hints": [
        "web_fetch"
      ],
      "confidence": 0.9
    },
    {
      "index": 2,
      "assigned_agent": "research",
      "description": "Extract citations + confidence notes for document",
      "dependencies": [
        "4c0c8413..."
      ],
      "tool_hints": [
        "web_fetch"
      ],
      "confidence": 0.9
    },
    {
      "index": 3,
      "assigned_agent": "reflection",
      "description": "Run reflection pass on aggregated outputs; sign-off or trigger replan.",
      "dependencies": [
        "bee9408d...",
        "4c0c8413...",
        "aab1a789..."
      ],
      "tool_hints": [],
      "confidence": 0.85
    }
  ],
  "metadata": {
    "promotion_report": {
      "tier_counts": {
        "T1": 5,
        "T2": 0,
        "T3": 0,
        "T4": 0,
        "T5": 0,
        "T6": 0
      }
    },
    "provenance_tail_sha": "0000000000000000000000000000000000000000000000000000000000000000",
    "total_events": 0
  }
}
```

Appendix C: Selected API Exchanges

login ok — HTTP 200, 217.6 ms (measured)

```
POST /v1/auth/login
{"user_id": "demo", "password": "Demo!12345678"}
-
{
  "access_token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJub2VzaXM6ZGV2ZWxvcG1lbnQlClJhdWQiOiJub2VzaXM6YXBPiIiwic3ViIjoizG
  VtbYsImp0aSI6IjZkY2YjEwYjg5ODg5OTQwMDNiZjg3YTZmZmNTI0NGE1NTdlIiwiaWF0IjoxNzNkxMjk2NzQ5LCJyYmYiOiE3OTY0Y3NDksImV4cC
  I6MTc5MTI5NzY0OSwicm9sZXMiOiJldjI0LCJ0b2t1b191c2UiOiJhY2NlCl3MifQ.WoL4W43byF-Ju3BEI6mPc-M-
  DLP6jtcpztA8JXOMSxw",
  "refresh_token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJub2VzaXM6ZGV2ZWxvcG1lbnQlClJhdWQiOiJub2VzaXM6YXBPiIiwic3ViIjoizG
  VtbYsImp0aSI6IjdmZTQzNDM0YjYjYjYzRjOWY4YThkOTAYNTQ0NWQ3MDC3IiwiaWF0IjoxNzNkxMjk2NzQ5LCJyYmYiOiE3OTY0Y3NDksImV4cC
  I6MTc5MTI5NzY0OSwicm9sZXMiOiJldjI0LCJ0b2t1b191c2UiOiJyZWZyZXNoIn0.-
  _q3qkZ06crgewoQj1YMBobzkxhabpIUzTG40NCrI2c",
  "token_type": "Bearer",
  "expires_in_s": 900
}
```

conversation create — HTTP 200, 1731.4 ms (measured)

```
POST /v1/conversations
{"title": "Write a Python function parse_csv.py that parses CSV files and add unit tests", "agent": "orchestrator", "tags": ["audit"]}
-
{
  "ok": true,
  "data": {
    "id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
    "title": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "status": "active",
    "agent": "orchestrator",
    "preview": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "tags": [
      "audit"
    ],
    "updated_at": "2026-10-06T14:25:58.037657Z",
    "created_at": "2026-10-06T14:25:56.318403Z",
    "message_count": 1,
    "token_count": 0,
    "cost_usd": 0.0,
    "user_id": null
  },
  "error": null,
  "request_id": "51a131deac9e457c9977bb5aa1b3619b",
  "meta": {
    "plan_id": "f207bd06-d02e-5459-b9be-0a35a633adfb",
    "run_id": "bb087c9c-cc2e-49ad-9ee7-f345e3ad2f8f"
  }
}
```

runs execute — HTTP 200, 1756.2 ms (measured)

```
POST /v1/runs/execute
{"plan": {"id": "087b732a-c670-58bf-8a63-4745d0ec9dbf", "goal": "Write a Python function parse_csv.py that parses CSV files and add unit tests", "steps": [{"id": "948796c3-0b7b-5dda-a5cb-d6b48aea1b3b", "index": 0, "description": "Draft implementation for parse_csv", "assigned_agent": "coding", "depe
-
{
  "_truncated_": true,
  "preview": "{\"ok\": true, \"data\": {\"run_id\": \"5ec84288-96aa-48f4-9263-ae836e52acf4\", \"steps\": [
[\"step_id\": \"948796c3-0b7b-5dda-a5cb-d6b48aea1b3b\", \"agent_type\": \"coding\", \"status\": \"SUCCESS\",
\"status_code\": 3, \"duration_ms\": 4.011900047771633, \"tokens_in\": 0, \"tokens_out\": 0, \"cost_usd\": 0.0,
\"result_preview\": \"coding: wrote new file (35 bytes, 1
lines)\\nwritten_content_sha256=6994726448f07cd3a7ff347b3a829cfc9cbd45451ac04b6ba2a918c2437187b4\\n\",
\"report_type\": \"CodingPatchReport\", \"report\": {\"target_file\": \"parse_csv.py\", \"plan_steps\": [\"1.
Read the current source with FilesTool (read operation).\", \"2. Identify exact snippet(s) to replace (preserve
surrounding context).\", \"3. Produce per-hunk PatchStep(s) with start_line_hint for line-number fidelity.\",
\"3b. Add tests/unit/test <feature>.py - at least 2 assert
```

llm benchmark deny mask — HTTP 403, 1.4 ms (measured)

GET /llm/benchmark

```
-
{
  "ok": false,
  "data": null,
  "error": "C1_DENY_MASK: deny mask 'llm.*' matched required capability 'llm.benchmark.read' (negative capability precedence: deny always wins)",
  "request_id": "c92dfba0009c4e218ad3e439bf45e0c6",
  "meta": {}
}
```

readyz — HTTP 200, 2037.6 ms (measured)

```
GET /health/readyz
-
{
  "ok": true,
  "data": {
    "status": "warn",
    "checks": {
      "sql": {
        "name": "sql",
        "status": "pass",
        "latency_ms": 19.9,
        "error": null
      },
      "redis": {
        "name": "redis",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "Error 22 connecting to 127.0.0.1:6379. The remote computer refused the network connection."
      },
      "qdrant": {
        "name": "qdrant",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "No module named 'qdrant_client'"
      }
    }
  },
  "error": null,
  "request_id": "28b1fe86c7e24f8a86e29cd8f511a5b0",
  "meta": {}
}
```

Appendix D: Configuration Reference

Table D.1: Environment variables

Variable	Meaning	Default
APP_ENV	development staging production	development
APP_NAME / APP_VERSION	Display metadata	Noesis / 0.1.0
APP_HOST / APP_PORT	Bind address (CLI serve)	0.0.0.0 / 8000
DEBUG	SQL echo; forbidden outside development	false
CORS_ORIGINS	CSV or JSON list; '*' rejected	http://localhost:3000
EVAL_ROOT	Benchmark tables root	docs/eval
SECRET_KEY	JWT signing secret (≥ 32 chars recommended)	development placeholder
JWT_ALGORITHM / JWT_EXPIRE_MINUTES	Declared; JWTService uses HS256 with fixed 15 min / 14 d	HS256 / 1440
RATE_LIMIT_PER_MINUTE	Declared; login/refresh limits are hard-coded	60
CLAIM_SUITES_HMAC_SECRET	Capability-token HMAC key	dev-only placeholder
DATABASE_URL	SQLAlchemy async URL	sqlite+aiosqlite:///./data/noesis.db
QDRANT_URL / QDRANT_API_KEY / QDRANT_COLLECTION_PREFIX	Vector store client	http://localhost:6333 / – / noesis
REDIS_URL	Cache client	redis://localhost:6379/0
DEFAULT_PROVIDER / FALLBACK_PROVIDER	openai anthropic gemini ollama openrouter llamacpp	openai ollama
DEFAULT_MODEL / TEMPERATURE / MAX_TOKENS / TIMEOUT	Generation defaults	gpt-4o-mini / 0.2 / 4096 / 120 s
EMBEDDING_PROVIDER / EMBEDDING_MODEL / EMBEDDING_DIMENSIONS	Embedding defaults	openai / text-embedding-3-small / 1536
OPENAI_API_KEY / OPENAI_BASE_URL / OPENAI_ORG_ID	OpenAI credentials	—
ANTHROPIC_API_KEY / ANTHROPIC_BASE_URL	Anthropic credentials	—
GOOGLE_API_KEY	Gemini key	—
OPENROUTER_API_KEY / OPENROUTER_BASE_URL	OpenRouter	— / https://openrouter.ai/api/v1
OLLAMA_BASE_URL / OLLAMA_MODEL	Ollama server and default model	http://localhost:11434 / llama3.1:8b
AGENTS_OLLAMA_MODEL	Model used for agent enrichment	qwen2.5-coder:7b-instruct-q4_K_M
LLAMACPP_BASE_URL / LLAMACPP_MODEL	llama.cpp server	http://localhost:8080 / llama-3
BACKEND_BASE_URL (frontend)	Rewrite target for /api/backend	http://localhost:8000
NEXT_PUBLIC_API_BASE (frontend)	Client API base	/api/backend

Appendix E: Agent Codenames

Table E.1: Codename mappings found in the code

AgentType	Backend prompt	Frontend label	Seeded pipeline
planner	Manan	Manan (Planner)	Manan
research	Anveshak	Anveshak (Researcher)	Darshak
coding	Vidya	Vidya (Coder)	Karmakarta
reflection	Darshak	— (Darshak is 'Analyst')	Vivechak
memory	Paalak	Paalak (Memory)	Paalak
rag	Vivechak-II	— (no RAG role)	Vidya
tool	Karmakarta	Karmakarta (Tooler)	Anveshak
judge	Parikshak	— (Parikshak is 'Tester')	Parikshak
critic	Vivechak	Vivechak (Critic)	Rakshak
executor	Kriyakārī	Kriyakārī (Executor)	Kriyakārī
supervisor	Nirikshak	Nirikshak (Supervisor)	Nirikshak
orchestrator	Samanyaka	— (Samanyakā is 'Synthesizer'; Rakshak is 'Security')	Samanyaka

Appendix F: Reconciliation of Earlier Project Claims

Table F.1: Claims versus implementation

Earlier claim	Finding
Tests '309/309 pytest' and '105/105 vitest'	312 collected: 310 passed, 2 xfailed; Vitest 103/105 on the working tree
'43 FastAPI routes'	42 operations on 36 paths + hidden GET /
'/llm/benchmark POST'	GET
Routes /v1/uap/cap-token/mint/verify, /v1/kernel/runs, /v1/kernel/scheduler, /v1/kernel/alloc, /v1/observability/spans	Do not exist
'JWT · CSRF · CORS' and auth on /v1 routes	JWT only on /v1/auth/me; no CSRF mechanism
'Kernel capability gate 5-stage HMAC on every tool call and spawn'	HMAC gate only on two HTTP routes; tools/spawns use in-process capability checks (no HMAC)
Memory tiers Indriya, Kushalata, Gyān, Ranniti, Yojanā, Tattva with Redis/Qdrant/Git backends	Code: T1 Indriya, T2 Smriti, T3 Gyān, T4 Yojanā, T5 Sāadhanā, T6 Paalak — all in-memory dicts
'Efron BCa' bootstrap	Percentile bootstrap (10 000 resamples)
SE-50/HumanEval/MBPP pass@1 for GPT-4o 1.000, Claude 0.987, Gemini 0.973	No result files for hosted models in the repository
'Ablation +18.0 pp'	/api/bench/results demo payload shows 0.22/0.31/0.40; no ablation run artefacts found
'LangGraph 0.6 StateGraph orchestration'	LangGraph declared as dependency but never imported
'9 whitelist tool adapters wired'	9 adapters exist; 4 in the default registry
'Determinism 450/450 SHA-256'	450 counts ordered pairs including self-pairs (50 goals × 3 ²); 50/50 goals identical
'12 pages + Bench Hub, 10 sidebar items'	13 routes in the working tree, 10 sidebar items
Jetson Orin Nano deployment, power/TCO figures (synopsis/PPT)	No Jetson-specific code or measurements
'42 % longer task retention' (PPT context)	No supporting data

Appendix G: Reproduction Commands

```
# backend tests (from backend/)
python -m pytest -q --cov=noesis
# run the API
uvicorn noesis.api.main:app --port 8000
# frontend checks (from frontend/)
npx vitest run ; npx tsc --noEmit ; npx eslint . --max-warnings=0
# planner determinism (Python)
from noesis.services.kernel_services import PlannerSvc
svc = PlannerSvc(); [svc.plan(goal, seed=42).plan.model_dump_json() for _ in range(3)]
# local LLM
ollama pull qwen2.5-coder:7b-instruct-q4_K_M
# evaluation harness (from backend/)
python scripts/run_w7_weekend.py --help
```